

LAPORAN TUGAS PROJEK
ALGORITMA PEMROGRAMAN – STATISTIKA A



Disusun Oleh :
KELOMPOK C

- | | |
|-------------------------------|--------------------|
| 1. RAHMAT TEJA BASKARA | (K1D025006) |
| 2. HILYA NAURA SAFLINA | (K1D025027) |
| 3. ROSA ALIFAH | (K1D025028) |
| 4. AVONA FEBRARY | (K1D025031) |

PROGRAM STUDI S-1 STATISTIKA
MATA KULIAH ALGORITMA PEMROGRAMAN
UNIVERSITAS JENDERAL SOEDIRMAN
TAHUN 2026

DAFTAR ISI

DAFTAR ISI	i
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Project.....	1
1.3 Ruang Lingkup Project	1
BAB II ANALISIS MASALAH.....	2
2.1 Program 1 - Menghitung Jumlah Kata dan Jumlah Kalimat	2
2.2 Program 2 – Pembuatan Objek String K1 Sampai K4	3
2.3 Program 3 - Menghitung Akar Persamaan Kuadrat	4
2.4 Program 4 - Menentukan NIP Karyawan	4
2.5 Program 5 - Menentukan Cluster Suatu Titik.....	5
2.6 Program 6 - Menentukan Interval Kepercayaan	6
BAB III PSEUDOCODE DAN FLOWCHART	9
3.1 Pseudocode	9
3.2 Flowchart.....	14
BAB IV SOURCE CODE	20
4.1 Source Code Python	20
4.2 Source Code R.....	26
BAB V PENJELASAN PROGRAM	31
5.1 Penjelasan Line-by-Line-Code R	31
5.2 Penjelasan Line-by-Line-Code Python	36
BAB VI PENGUJIAN PROGRAM.....	43
6.1 Tabel Pengujian.....	43
6.2 Screenshot Hasil Running	45
6.3 Interpretasi Output.....	63
BAB VII PENUTUP.....	68
7.1 Kesimpulan.....	68
7.2 Saran.....	68
Lampiran.....	69

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kemampuan pemrograman merupakan salah satu keterampilan yang penting untuk dimiliki mahasiswa di era perkembangan teknologi dan data saat ini. Melalui pemrograman, berbagai permasalahan dapat diselesaikan secara lebih cepat, tepat, dan sistematis. Oleh karena itu, pemahaman mengenai algoritma sebagai langkah-langkah penyelesaian masalah menjadi dasar yang harus dikuasai sebelum mengembangkan sebuah program.

Pada mata kuliah Algoritma dan Pemrograman, mahasiswa tidak hanya dituntut untuk memahami teori, tetapi juga mampu menerapkannya dalam bentuk program yang dapat dijalankan. Project ini disusun sebagai sarana untuk mengasah kemampuan berpikir logis, analitis, dan terstruktur dalam menyelesaikan berbagai permasalahan komputasional. Melalui enam studi kasus yang berbeda, mahasiswa diharapkan dapat memahami bagaimana suatu masalah diterjemahkan menjadi algoritma, kemudian diimplementasikan menggunakan bahasa pemrograman Python dan R.

Selain aspek teknis pemrograman, project ini juga memberikan pengalaman dalam menyusun dokumentasi program, melakukan pengujian, serta bekerja sama dalam tim. Dengan demikian, project ini diharapkan dapat menjadi media pembelajaran yang membantu mahasiswa memahami konsep algoritma dan pemrograman secara lebih mendalam dan aplikatif.

1.2 Tujuan Project

1. Meningkatkan pemahaman mengenai konsep dasar algoritma dan pemrograman.
2. Melatih kemampuan dalam menyusun solusi masalah melalui pseudocode dan flowchart.
3. Menerapkan konsep pemrograman menggunakan bahasa Python dan R.
4. Melatih kemampuan dalam menguji dan mengevaluasi hasil program yang dibuat.
5. Mengembangkan kemampuan kerja sama dan komunikasi dalam menyelesaikan project kelompok.

1.3 Ruang Lingkup Project

1. Meningkatkan pemahaman mengenai konsep dasar algoritma dan pemrograman.
2. Melatih kemampuan dalam menyusun solusi masalah melalui pseudocode dan flowchart.
3. Menerapkan konsep pemrograman menggunakan bahasa Python dan R.
4. Melatih kemampuan dalam menguji dan mengevaluasi hasil program yang dibuat.
5. Mengembangkan kemampuan kerja sama dan komunikasi dalam menyelesaikan project kelompok.

BAB II ANALISIS MASALAH

2.1 Program 1 – Menghitung Jumlah Kata dan Jumlah Kalimat

A. Uraikan Soal

Program ini dibuat untuk menghitung jumlah kata dan jumlah kalimat dari sebuah teks yang dimasukkan oleh pengguna. Pada soal ini diasumsikan bahwa tanda titik (.) hanya digunakan sebagai penanda akhir kalimat sehingga jumlah kalimat dapat dihitung berdasarkan banyaknya tanda titik yang terdapat pada teks.

B. Tabel Input-Proses-Output

INPUT	PROSES	OUTPUT
Teks atau paragraf yang dimasukkan oleh pengguna	1. Menerima input teks dari pengguna. 2. Memisahkan teks menjadi kata-kata berdasarkan spasi. 3. Menghitung jumlah kata dalam teks. 4. Menghitung jumlah tanda titik (.) sebagai jumlah kalimat. 5. Menyimpan hasil perhitungan.	Jumlah kata dan jumlah kalimat yang terdapat pada teks.

C. Batasan Masalah

Pada program ini, pengguna diminta memasukkan sebuah teks atau paragraf yang akan dianalisis. Jumlah kata dihitung berdasarkan pemisahan menggunakan spasi, sedangkan jumlah kalimat dihitung berdasarkan banyaknya tanda titik (.) yang terdapat pada teks. Program ini tidak memperhitungkan tanda baca lain seperti tanda tanya (?) atau tanda seru (!) sebagai akhir kalimat. Selain itu, program juga tidak memeriksa apakah susunan kalimat sudah sesuai dengan aturan tata bahasa yang benar, melainkan hanya berfokus pada perhitungan jumlah kata dan jumlah kalimat.

D. Kondisi Khusus

Program dirancang agar tetap dapat berjalan pada beberapa kondisi yang berbeda. Jika pengguna memasukkan satu kalimat, program akan menghitung jumlah kata dan jumlah kalimat yang terdapat pada kalimat tersebut. Jika pengguna memasukkan beberapa kalimat dalam satu paragraf, program akan menghitung seluruh kata dan jumlah kalimat yang ada. Apabila pengguna tidak memasukkan teks sama sekali, program akan menampilkan pesan bahwa teks tidak boleh kosong. Selain itu, jika teks tidak memiliki tanda titik, maka jumlah kalimat yang dihitung adalah nol.

karena program menggunakan tanda titik sebagai penanda akhir kalimat. Dengan demikian, program tetap dapat memberikan hasil sesuai dengan kondisi input yang diberikan oleh pengguna.

2.2 Program 2 – Pembuatan Objek String K1 sampai K4

A. Uraikan Soal

Pada soal ini, program dibuat untuk menampilkan empat buah kalimat yang disimpan dalam variabel K1, K2, K3, dan K4. Setiap kalimat memiliki format yang berbeda, terutama dalam penggunaan tanda petik tunggal dan tanda petik ganda. Oleh karena itu, program harus ditulis dengan benar agar seluruh kalimat dapat ditampilkan sesuai dengan yang diminta pada soal. Program ini tidak memerlukan input dari pengguna karena semua kalimat sudah ditentukan sebelumnya. Program hanya perlu membuat empat variabel string, kemudian menampilkan isi dari masing-masing variabel tersebut ke layar. Melalui soal ini, mahasiswa dapat memahami cara membuat variabel bertipe string, menggunakan karakter khusus dalam string, serta menampilkan output sesuai format yang diinginkan.

B. Tabel Input-Proses-Output

INPUT	PROSES	OUTPUT
Tidak ada	Membuat empat variabel string, yaitu K1, K2, K3, dan K4 sesuai dengan kalimat yang telah ditentukan. Selanjutnya program menampilkan isi dari masing-masing string ke layar.	Empat kalimat yang tersimpan dalam variabel K1, K2, K3, dan K4 ditampilkan sesuai format yang diminta pada soal.

C. Batasan Masalah

Pada program ini, pengguna tidak perlu memasukkan data apa pun karena semua kalimat yang akan ditampilkan sudah ditentukan di dalam program. Program hanya bertugas membuat empat variabel string, yaitu K1, K2, K3, dan K4, lalu menampilkannya ke layar. Selain itu, isi kalimat yang ditampilkan harus sama persis dengan yang terdapat pada soal, termasuk penggunaan tanda petik dan tanda baca lainnya.

D. Kondisi Khusus

Program harus dapat menampilkan kalimat yang mengandung tanda petik tunggal maupun tanda petik ganda dengan benar. Karena tidak ada input dari pengguna dan tidak terdapat proses perhitungan, hasil yang ditampilkan akan selalu sama setiap kali program dijalankan. Oleh karena itu, hal yang perlu diperhatikan adalah penulisan string harus tepat agar output yang dihasilkan sesuai dengan yang diminta pada soal.

2.3 Program 3 - Menghitung akar persamaan kuadrat

A. Uraikan Soal

Program ini meminta kita untuk membuat sebuah sistem komputerisasi yang dapat menghitung akar-akar dari sebuah persamaan kuadrat $ax^2 + bx + c = 0$ menggunakan Rumus ABC.

Sistem harus menerima masukan berupa tiga koefisien (a, b, dan c) lalu menganalisis jenis akarnya berdasarkan nilai Diskriminan (D). Jika akar-akarnya riil, program akan menampilkan hasilnya dengan ketelitian 3 angka di belakang desimal. Jika akarnya imajiner, program cukup menampilkan pesan bahwa persamaan tersebut hanya memiliki akar imajiner.

B. Tabel input-Proses-Output

INPUT	PROSES	OUTPUT
Nilai koefisien persamaan kuadrat yaitu nilai a, b dan c	1. Menghitung nilai diskriminan ($D = b^2 - 4.a.c$) 2. Logika percabangan untuk nilai $D > 0$, $D = 0$, $D < 0$ (If, Else If, Else)	Nilai akar kuadrat dengan bilangan real 3 desimal yang berbeda atau nilai akar kembar atau "Akar-akar bersifat imajiner"

C. Batasan Masalah

1. Format Output: Akar bilangan riil wajib ditampilkan tepat dalam tiga angka di belakang desimal.
2. Kondisi Imajiner: Jika $D < 0$, program tidak perlu (dan tidak boleh) memaksakan perhitungan akar kuadrat negatif karena akan menyebabkan error *NaN (Not a Number)* pada program, melainkan langsung dialihkan ke output teks.

D. Kondisi Khusus

1. Nilai $a = 0$: Jika pengguna memasukkan $a = 0$, persamaan tersebut bukan lagi persamaan kuadrat melainkan persamaan linear ($bx + c = 0$). Dalam rumus ABC, nilai a berada di posisi pembagi ($2a$). Jika $a = 0$, akan terjadi fatal error berupa *Division by Zero* (pembagian dengan angka nol). Program yang baik harus memvalidasi agar a tidak boleh sama dengan 0.
2. Nilai $D = 0$: Meskipun masuk dalam kategori akar riil, nilai x_1 dan x_2 akan menghasilkan angka yang sama. Program dapat dioptimalkan untuk hanya menampilkan satu nilai x saja agar tampilan lebih efisien.

2.4 Program 4 - Menentukan NIP Karyawan

A. Uraikan Soal

Program ini dibuat untuk menentukan tanggal lahir ASN berdasarkan Nomor Induk Pegawai (NIP) yang dimasukkan oleh pengguna. Pada NIP ASN, delapan digit pertama menyatakan informasi tanggal lahir, yaitu empat digit pertama menunjukkan

tahun lahir, dua digit berikutnya menunjukkan bulan lahir, dan dua digit selanjutnya menunjukkan tanggal lahir. Program harus membaca NIP yang dimasukkan, mengambil informasi tahun, bulan, dan tanggal lahir dari delapan digit pertama tersebut, kemudian menampilkan tanggal lahir ASN dalam format yang lebih mudah dibaca, yaitu tanggal, nama bulan, dan tahun.

B. Tabel input-Proses-Output

INPUT	PROSES	OUTPUT
NIP ASN yang terdiri 18 digit.	Memeriksa panjang NIP, mengambil 4 digit pertama sebagai tahun, 2 digit berikutnya sebagai bulan, 2 digit berikutnya sebagai tanggal, lalu mengubah kode bulan menjadi nama bulan dan memvalidasi tanggal/bulan	Tanggal lahir ASN dalam format tanggal nama bulan tahun, atau pesan kesalahan jika input tidak valid.

C. Batasan Masalah

- NIP yang diproses harus terdiri dari 18 digit.
- Program hanya menggunakan 8 digit pertama NIP untuk menentukan tanggal lahir.
- Format delapan digit pertama NIP diasumsikan YYYYMMDD.
- Nilai bulan yang dianggap valid hanya 01 sampai 12.
- Nilai tanggal yang dianggap valid hanya 01 sampai 31.
- Program hanya menampilkan tanggal lahir ASN, bukan memproses informasi lain pada NIP.

D. Kondisi Khusus

- Jika panjang NIP tidak sama dengan 18 digit, program menampilkan pesan bahwa NIP tidak valid.
- Jika kode bulan tidak berada pada rentang 01–12, program menampilkan pesan bahwa bulan pada NIP tidak valid.
- Jika nilai tanggal tidak berada pada rentang 01–31, program menampilkan pesan bahwa tanggal pada NIP tidak valid.
- Jika input mengandung karakter yang tidak sesuai format NIP, maka input dianggap tidak valid dan program menampilkan pesan kesalahan.

2.5 Program 5 - Menentukan Cluster Suatu Titik

A. Uraikan Soal

Pada soal ini diberikan tiga titik yang telah ditetapkan sebagai pusat cluster pada ruang tiga dimensi, yaitu Cluster A dengan koordinat $A(2,1,3)$, Cluster B dengan koordinat $B(1,-4,6)$, dan Cluster C $(-2,3,-2)$. Selanjutnya kita masukkan sebuah titik sembarang yaitu $U(x_1, x_2, x_3)$. Program bertugas menghitung jarak titik U terhadap masing-masing pusat cluster menggunakan rumus Euclidean tiga dimensi. Setelah diperoleh jarak dari titik U menuju tiga pusat cluster, program akan membandingkan

ketiga hasil tersebut dan titik U akan diklasifikasikan ke dalam cluster yang memiliki jarak yang paling kecil. Di Akhir program akan menampilkan jarak titik U ke masing-masing cluster serta hasil klasifikasi dari titik tersebut termasuk cluster A, cluster B, atau cluster C.

B. Tabel Input-Proses-Output

INPUT	PROSES	OUTPUT
Koordinat Titik U berupa x_1 , x_2 , dan x_3 serta harus berupa angka.	1. Mendefinisikan atau menetapkan pusat Cluster A, B, dan C. 2. Menghitung jarak titik U ke masing-masing cluster. 3. Membandingkan ketiga jarak. 4. Menentukan cluster dengan jarak terkecil.	Nilai jarak ke Cluster A, B, dan C serta hasil klasifikasi titik U sebagai anggota Cluster A, B, atau C.

C. Batasan Masalah

- a. Program hanya menggunakan tiga cluster , yaitu Cluster A, Cluster b, dan Cluster C.
- b. Pusat Cluster bersifat tetap, yaitu $A = (2, 1, 3)$, $B = (1, -4, 6)$, $C = (-2, 3, -2)$.
- c. Titik yang akan diklasifikasikan hanya terdiri atas tiga koordinat (x_1 , x_2 , x_3)
- d. Perhitungan jarak menggunakan rumus euclidean tiga dimensi.
- e. Program hanya menentukan cluster berdasarkan jarak terdekat.

D. Kondisi Khusus

- a. Jika jarak ke Cluster A paling kecil, maka titik U diklasifikasikan sebagai anggota Cluster A.
- b. Jika jarak ke Cluster B paling kecil, maka titik U diklasifikasikan sebagai anggota Cluster B.
- c. Jika jarak ke Cluster C paling kecil, maka titik U diklasifikasikan sebagai anggota Cluster C.
- d. Jika terdapat dua atau tiga jarak yang sama besar (jarak minimum sama), maka pada program yang dibuat hasil cluster mengikuti urutan pemeriksaan pada percabangan (if-else if-else) sehingga cluster yang diperiksa terlebih dahulu akan dipilih. Misalnya, jika jarak titik U ke pusat cluster A sama dengan jarak titik U ke pusat cluster B dan keduanya merupakan jarak minimum, maka program akan mengklasifikasikannya ke Cluster A.

2.6 Program 6 - Menentukan Interval Kepercayaan

A. Uraikan Soal

Pada soal ini diminta membuat program untuk menghitung interval kepercayaan proporsi populasi berdasarkan proporsi sampel, ukuran sampel, dan tingkat signifikansi (α). Perhitungan dilakukan menggunakan rumus interval kepercayaan proporsi.

Nilai z yang digunakan bergantung pada nilai α , yaitu 1,645 untuk $\alpha=10\%$ dan 1,96 untuk $\alpha=5\%$. Program menerima masukan berupa proporsi sampel, ukuran sampel, dan nilai α , kemudian menghitung batas bawah dan batas atas interval kepercayaan. Selain itu, program juga melakukan validasi terhadap nilai proporsi. Jika proporsi yang dimasukkan kurang dari 0 atau lebih dari 1, maka program menampilkan pesan kesalahan karena nilai tersebut tidak memenuhi definisi proporsi.

B. Tabel Input-Proses-Output

INPUT	PROSES	OUTPUT
Proporsi sampel, ukuran sampel, dan nilai α (0,10 atau 0,05).	1. Memvalidasi nilai proporsi. 2. Menentukan nilai z berdasarkan α . 3. Menghitung <i>standard error</i> . 4. Menghitung <i>margin of error</i> . 5. Menghitung batas bawah dan batas atas interval kepercayaan.	Interval kepercayaan proporsi populasi atau pesan kesalahan jika input tidak valid.

C. Batasan Masalah

- Program hanya menerima nilai $\alpha=0,10$ atau $\alpha=0,05$.
- Nilai proporsi sampel harus berada pada rentang $0 \leq \text{proporsi} \leq 1$.
- Ukuran sampel (n) harus berupa bilangan bulat positif ($n>0$).
- Perhitungan interval kepercayaan menggunakan rumus interval kepercayaan proporsi populasi.
- Program hanya menghitung interval kepercayaan dan tidak melakukan analisis statistik lainnya.

D. Kondisi Khusus

- Jika nilai proporsi kurang dari 0 atau lebih dari 1, program menampilkan pesan "Error: Proporsi harus berada antara 0 dan 1." dan proses dihentikan.
- Jika nilai α bukan 0,10 atau 0,05, program menampilkan pesan "Error: Alpha harus 0.10 atau 0.05." dan proses dihentikan.
- Jika seluruh input valid, program menghitung *standard error*, *margin of error*, serta batas bawah dan batas atas interval kepercayaan, kemudian menampilkan hasil interval kepercayaannya kepada pengguna.

- d. Hasil interval kepercayaan ditampilkan dalam bentuk (batas bawah, batas atas) dengan empat angka di belakang koma agar hasil lebih mudah dibaca dan lebih presisi.

BAB III

PSEUDOCODE DAN FLOWCHART

3.1 PSEUDOCODE

A. Pemrograman 1 :

START

INPUT teks

Simpan hasil pemisahan teks berdasarkan spasi
ke dalam variabel kata

Hitung banyaknya elemen pada variabel kata
Simpan sebagai jumlah_kata

Hitung banyaknya tanda titik (.) pada teks
Simpan sebagai jumlah_kalimat

OUTPUT jumlah_kata

OUTPUT jumlah_kalimat

END

B. Pemrograman 2 :

START

Buat string K1 berisi "Saya tak 'kan menyerah."

Buat string K2 berisi
"Ia berkata, "Aku menyayangimu.""

Buat string K3 berisi
""Coba jelaskan pengertian 'cross-validation'
dalam Machine Learning!""

Buat string K4 berisi
"Surat keputusan itu bernomor
62/UN.34/19/2023."

Tampilkan K1

Tampilkan K2

Tampilkan K3

Tampilkan K4

END

C. Pemrograman 3 :

Mulai

Input a

Input b

Input c

$D \leftarrow b^2 - 4ac$

Jika $D > 0$ maka

$x_1 \leftarrow (-b + \sqrt{D}) / (2a)$

$x_2 \leftarrow (-b - \sqrt{D}) / (2a)$

Tampilkan "Akar real berbeda"

Tampilkan x_1 dan x_2

Jika tidak, jika $D = 0$ maka

$x \leftarrow -b / (2a)$

Tampilkan "Akar real kembar"

Tampilkan x

Jika tidak maka

Tampilkan "Persamaan memiliki akar-akar imajiner"

Selesai

D. Pemrograman 4 :

Mulai

Input NIP

Jika panjang NIP $\neq 18$ maka

Tampilkan "NIP harus terdiri dari 18 digit"

Jika tidak maka

$\text{tahun} \leftarrow 4 \text{ digit pertama NIP}$

$\text{bulan} \leftarrow 2 \text{ digit ke-5 dan ke-6 NIP}$

tanggal ← 2 digit ke-7 dan ke-8 NIP

```
Jika bulan = "01" maka
nama_bulan ← "Januari"
Else jika bulan = "02" maka
nama_bulan ← "Februari"
Else jika bulan = "03" maka
nama_bulan ← "Maret"
Else jika bulan = "04" maka
nama_bulan ← "April"
Else jika bulan = "05" maka
nama_bulan ← "Mei"
Else jika bulan = "06" maka
nama_bulan ← "Juni"
Else jika bulan = "07" maka
nama_bulan ← "Juli"
Else jika bulan = "08" maka
nama_bulan ← "Agustus"
Else jika bulan = "09" maka
nama_bulan ← "September"
Else jika bulan = "10" maka
nama_bulan ← "Oktober"
Else jika bulan = "11" maka
nama_bulan ← "November"
Else jika bulan = "12" maka
nama_bulan ← "Desember"
Else
nama_bulan ← "Tidak Valid"
```

```
Jika nama_bulan = "Tidak Valid" maka
Tampilkan "Bulan pada NIP tidak valid"
```

```
Else jika tanggal < 1 atau tanggal > 31 maka
Tampilkan "Tanggal pada NIP tidak valid"
```

```
Else
Tampilkan "Tanggal lahir ASN : "
tanggal, nama_bulan, tahun
```

Selesai

E. Pemrograman 5 :

Mulai

Tentukan pusat cluster:

A = (2, 1, 3)

B = (1, -4, 6)

C = (-2, 3, -2)

Input x1, x2, x3

Simpan titik U = (x1, x2, x3)

Hitung jarakA = $\sqrt{[(x1 - 2)^2 + (x2 - 1)^2 + (x3 - 3)^2]}$

Hitung jarakB = $\sqrt{[(x1 - 1)^2 + (x2 + 4)^2 + (x3 - 6)^2]}$

Hitung jarakC = $\sqrt{[(x1 + 2)^2 + (x2 - 3)^2 + (x3 + 2)^2]}$

Jika jarakA < jarakB dan jarakA < jarakC maka

cluster = "A"

Else jika jarakB < jarakA dan jarakB < jarakC maka

cluster = "B"

Else

cluster = "C"

Tampilkan "Titik U berada pada Cluster", cluster

Selesai

F. Pemrograman 6 :

Mulai

Input p_hat

Input n

Input alpha

Jika p_hat < 0 atau p_hat > 1

Tampilkan "Error: proporsi harus antara 0 dan 1"

Selesai

Jika alpha = 0.10

$$z \leftarrow 1.645$$

Jika tidak, jika $\alpha = 0.05$

$$z \leftarrow 1.96$$

Selain itu

Tampilkan "Alpha tidak valid"

Selesai

Hitung SE

$$SE \leftarrow \sqrt{(p_hat \times (1-p_hat))/n}$$

Hitung batas bawah

$$Batas_bawah \leftarrow p_hat - z \times SE$$

$$\text{Hitung batas atas } batas_atas \leftarrow p_hat + z \times SE$$

Tampilkan "Interval Kepercayaan"

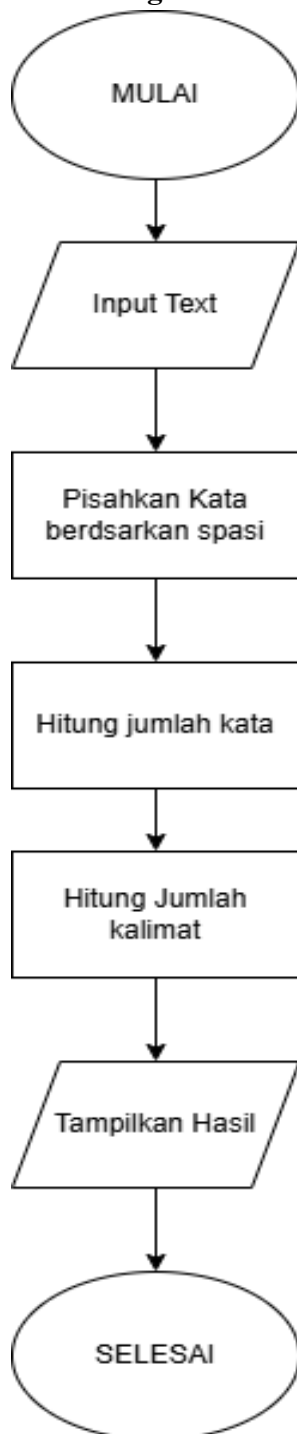
Tampilkan

$$batas_bawah < p < batas_atas$$

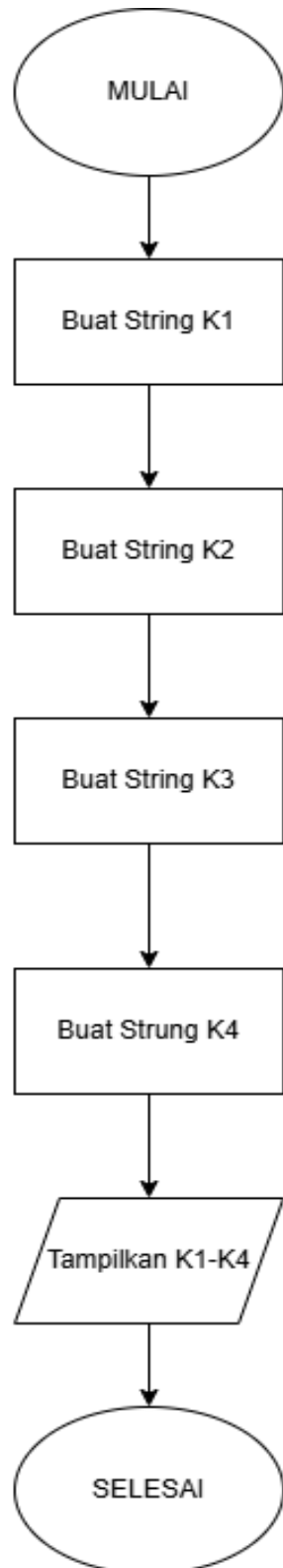
Selesai

3.2 FLOWCHART

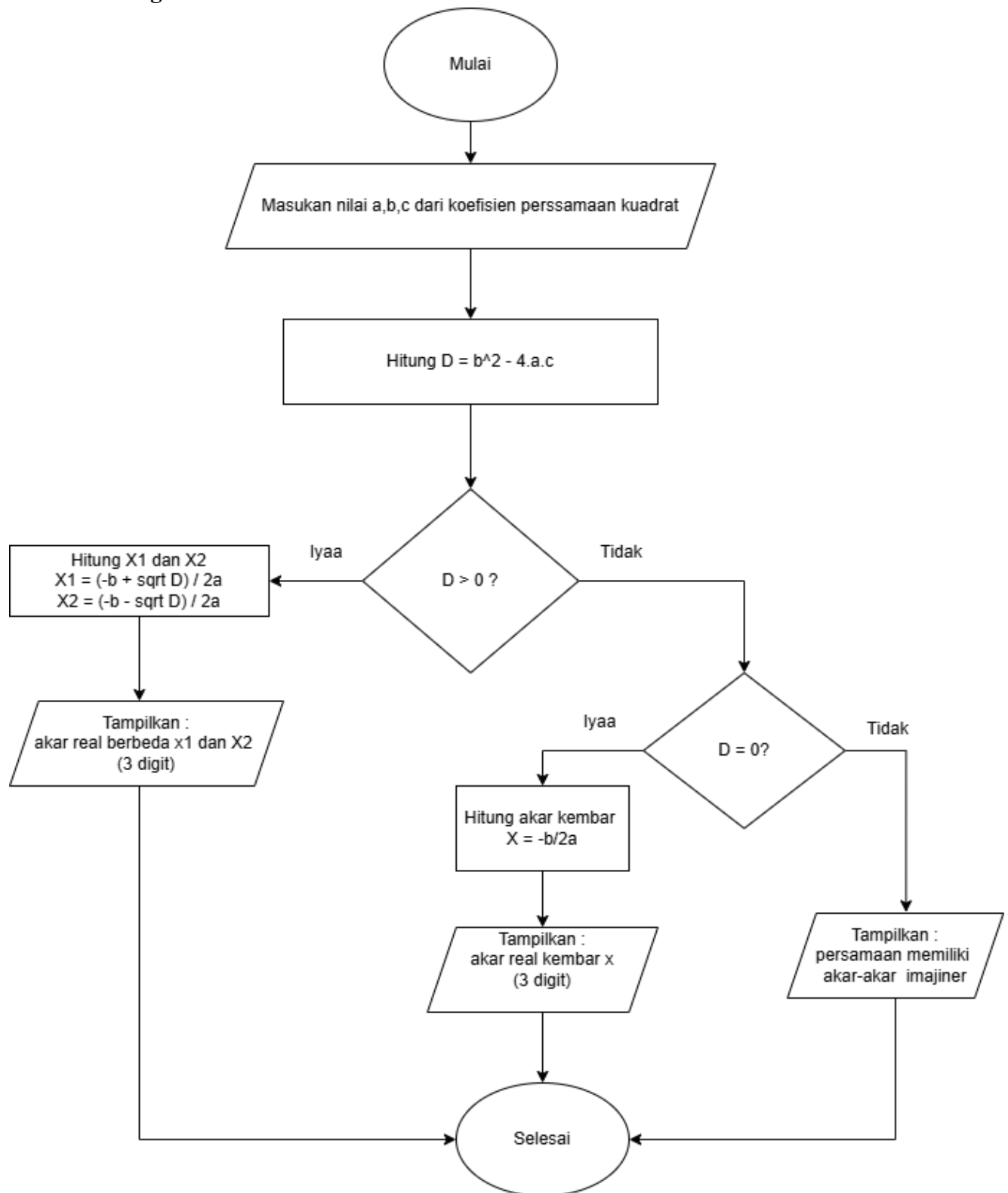
A. Pemrograman 1 :



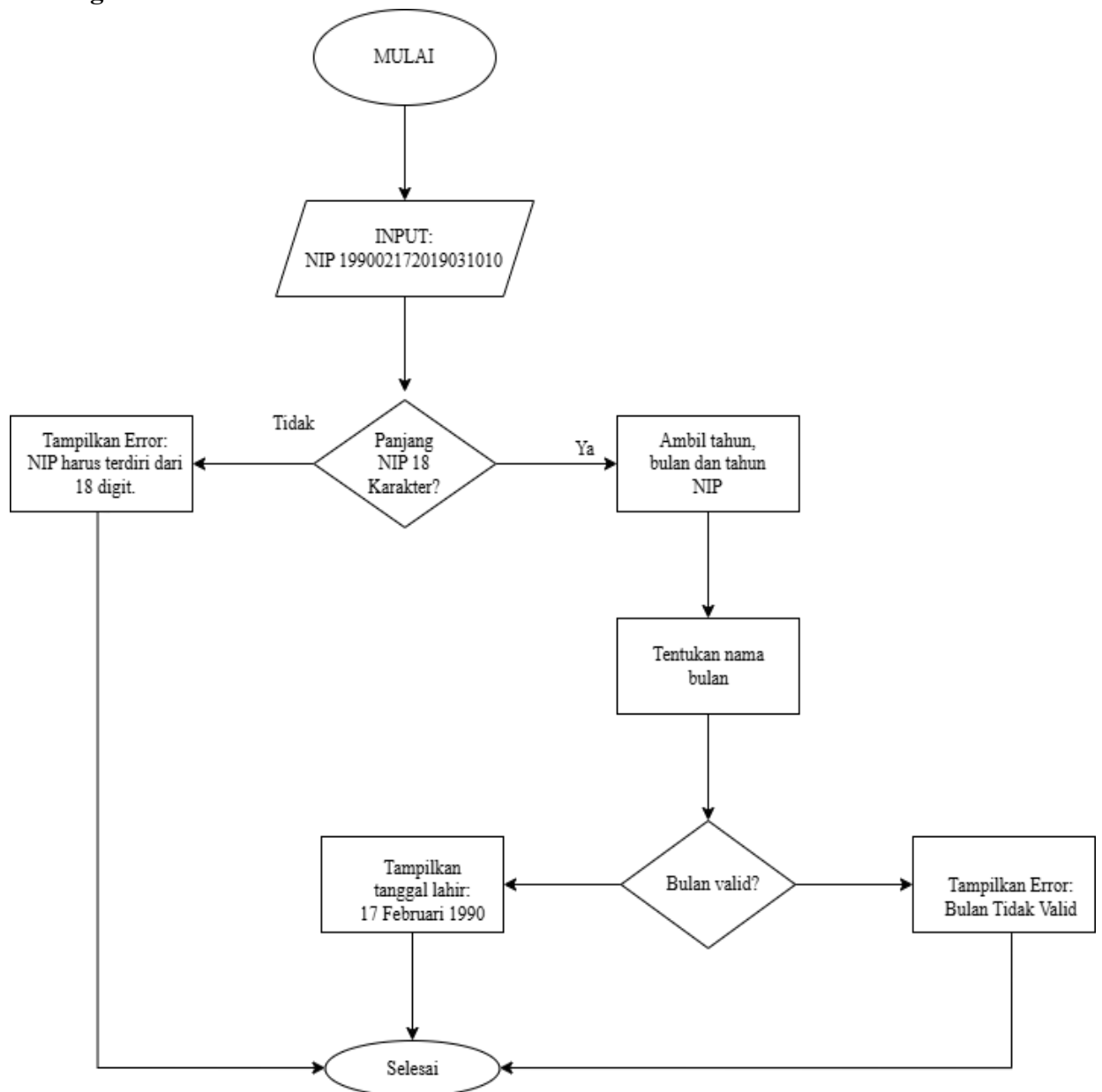
B. Pemrograman 2 :



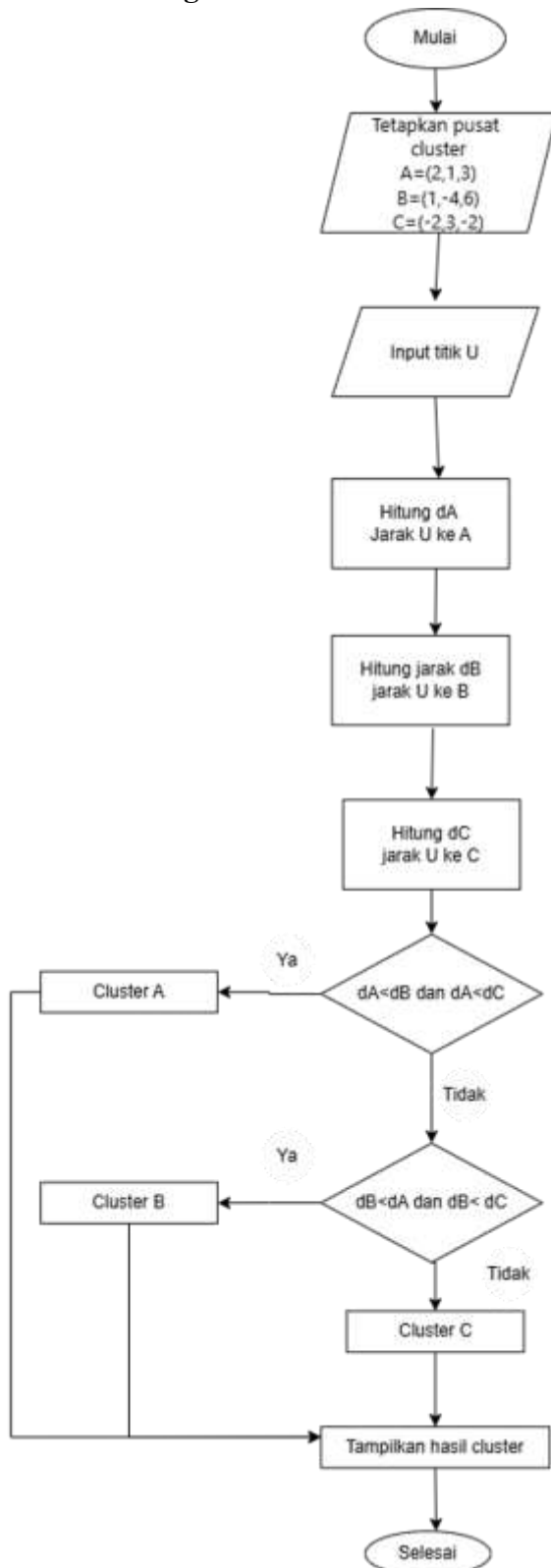
C. Pemrograman 3 :



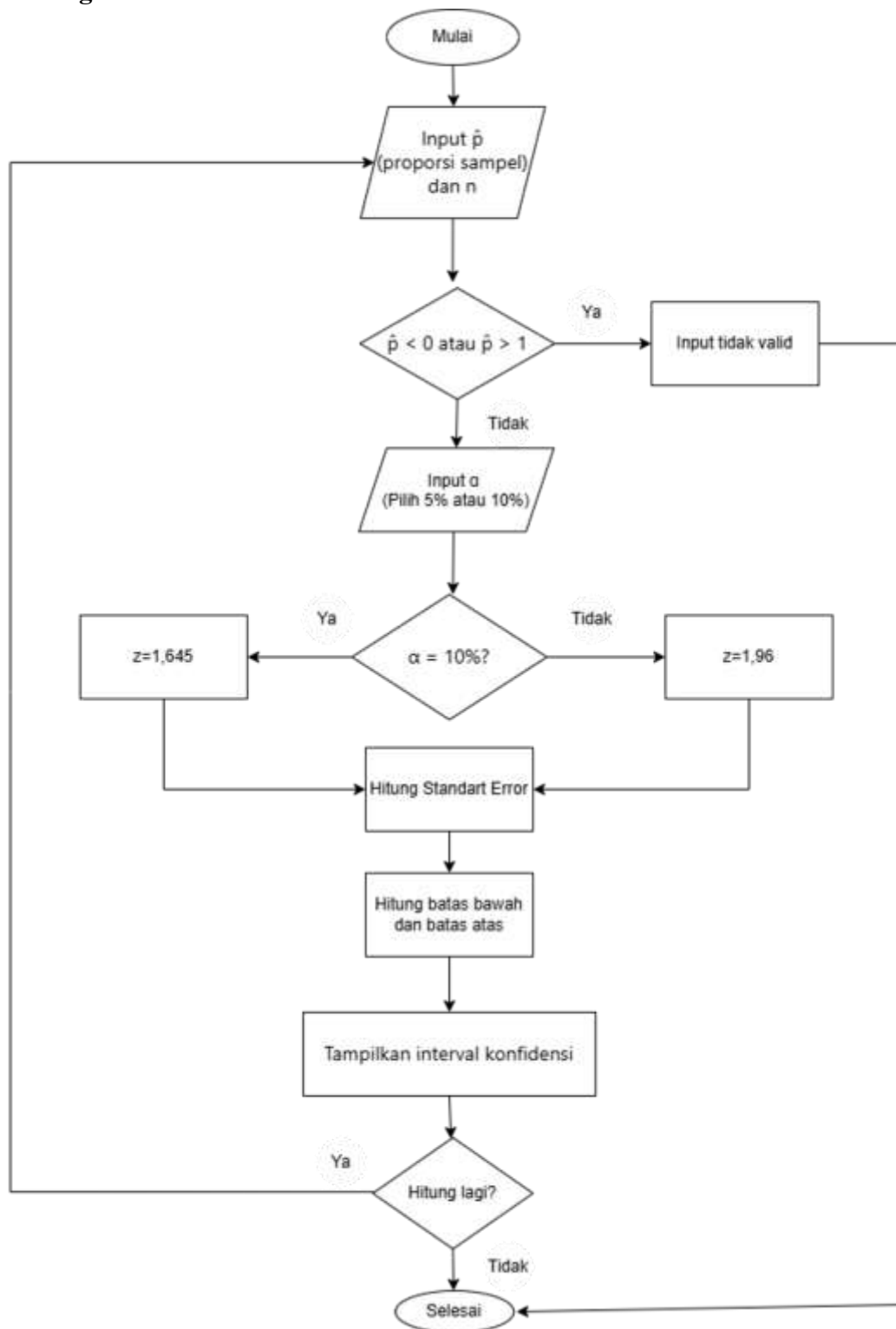
D. Pemrograman 4 :



E. Pemrograman 5 :



F. Pemrograman 6 :



BAB IV SOURCE CODE

4.1 Source Code Python

A. Pemrograman 1 :

```
# Soal Program 1
# Program Menghitung Jumlah Kata dan Jumlah Kalimat

# Membaca teks yang dimasukkan pengguna
teks = input("Masukkan teks: ")

# Memisahkan teks menjadi kata-kata berdasarkan spasi
kata = teks.split()

# Menghitung jumlah kata
jumlah_kata = len(kata)

# Menghitung jumlah tanda titik sebagai jumlah kalimat
jumlah_kalimat = teks.count(".")

# Menampilkan hasil perhitungan
print("\nHasil Perhitungan")
print("Jumlah kata :", jumlah_kata)
print("Jumlah kalimat :", jumlah_kalimat)
```

B. Pemrograman 2 :

```
# Soal Program 2
# Program 2 - Membuat objek string K1 sampai K4

# String yang mengandung tanda petik tunggal
K1 = "Saya tak 'kan menyerah."

# String yang mengandung tanda petik ganda
K2 = 'Ia berkata, "Aku menyayangimu."'

# String yang mengandung petik tunggal dan ganda
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
```

```
# String biasa
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."
```

```
# Menampilkan isi string
print(K1)
print(K2)
print(K3)
print(K4)
```

C. Pemrograman 3 :

```
# Program Mencari Akar Persamaan Kuadrat
```

```
a = float(input("Masukkan nilai a : "))
b = float(input("Masukkan nilai b : "))
c = float(input("Masukkan nilai c : "))
```

```
D = b**2 - 4*a*c
```

```
if D > 0:
```

```
    x1 = (-b + (D**0.5)) / (2*a)
```

```
    x2 = (-b - (D**0.5)) / (2*a)
```

```
print("Akar real berbeda")
```

```
print("x1 = %.3f" % x1)
```

```
print("x2 = %.3f" % x2)
```

```
elif D == 0:
```

```
    x = -b / (2*a)
```

```
print("Akar real kembar")
```

```
print("x1 = x2 = %.3f" % x)
```

```
else:
```

```
    print("Persamaan memiliki akar-akar imajiner")
```

D. Pemrograman 4 :

```
# Input NIP ASN
```

```
nip = input("Masukkan NIP ASN: ")
```

```

# Validasi panjang NIP
if len(nip) != 18:
    print("NIP harus terdiri dari 18 digit")

else:
    # Mengambil tahun, bulan, dan tanggal lahir
    tahun = nip[0:4]
    bulan = nip[4:6]
    tanggal = nip[6:8]

    bulan_num = int(bulan)
    tanggal_num = int(tanggal)

    # Menentukan nama bulan
    if bulan == "01":
        nama_bulan = "Januari"
    elif bulan == "02":
        nama_bulan = "Februari"
    elif bulan == "03":
        nama_bulan = "Maret"
    elif bulan == "04":
        nama_bulan = "April"
    elif bulan == "05":
        nama_bulan = "Mei"
    elif bulan == "06":
        nama_bulan = "Juni"
    elif bulan == "07":
        nama_bulan = "Juli"
    elif bulan == "08":
        nama_bulan = "Agustus"
    elif bulan == "09":
        nama_bulan = "September"
    elif bulan == "10":
        nama_bulan = "Oktober"
    elif bulan == "11":
        nama_bulan = "November"
    elif bulan == "12":

```



```

nama_bulan = "Desember"
else:
nama_bulan = "Tidak Valid"

# Validasi bulan dan tanggal
if nama_bulan == "Tidak Valid":
print("Bulan pada NIP tidak valid")

elif tanggal_num < 1 or tanggal_num > 31:
print("Tanggal pada NIP tidak valid")

else:
print("Tanggal lahir ASN:", tanggal, nama_bulan, tahun)

```

E. Pemrograman 5 :

#Soal nomor 5

```
import math
```

```

def hitung_jarak(titik1, titik2):
    return math.sqrt(
        (titik1[0] - titik2[0]) ** 2 +
        (titik1[1] - titik2[1]) ** 2 +
        (titik1[2] - titik2[2]) ** 2
    )

```

```

def tentukan_cluster(titik_u):
    cluster_a = (2, 1, 3)
    cluster_b = (1, -4, 6)
    cluster_c = (-2, 3, -2)

    jarak_a = hitung_jarak(titik_u, cluster_a)
    jarak_b = hitung_jarak(titik_u, cluster_b)
    jarak_c = hitung_jarak(titik_u, cluster_c)

    print(f'Jarak ke Cluster A = {jarak_a}')
    print(f'Jarak ke Cluster B = {jarak_b}')
    print(f'Jarak ke Cluster C = {jarak_c}')

```

```

if jarak_a <= jarak_b and jarak_a <= jarak_c:
    return "A"
elif jarak_b <= jarak_a and jarak_b <= jarak_c:
    return "B"
else:
    return "C"

try:
    x1 = float(input("Masukkan x1: "))
    x2 = float(input("Masukkan x2: "))
    x3 = float(input("Masukkan x3: "))

    titik_u = (x1, x2, x3)

    hasil = tentukan_cluster(titik_u)

    print("Titik U termasuk Cluster", hasil)

except ValueError:
    print("Input harus berupa angka!")

```

F. Pemrograman 6 :

```

import math
# Fungsi menentukan nilai z berdasarkan alpha
def nilai_z(alpha):
    if alpha == 0.10:
        return 1.645
    elif alpha == 0.05:
        return 1.96
    else:
        return None

# Fungsi menghitung interval kepercayaan
def interval_kepercayaan(p_hat, n, alpha):
    z = nilai_z(alpha)

```

```

margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)

batas_bawah = p_hat - margin_error
batas_atas = p_hat + margin_error

return batas_bawah, batas_atas

# Program utama
def main():
    print("=== Interval Kepercayaan Proporsi ===")

    p_hat = float(input("Masukkan proporsi sampel ( $\hat{p}$ ): "))
    n = int(input("Masukkan ukuran sampel (n): "))
    alpha = float(input("Masukkan alpha (0.10 atau 0.05): "))

    # Validasi proporsi
    if p_hat < 0 or p_hat > 1:
        print("Error: Proporsi harus berada antara 0 dan 1.")
        return

    # Validasi alpha
    if nilai_z(alpha) is None:
        print("Error: Alpha harus 0.10 atau 0.05.")
        return

    bawah, atas = interval_kepercayaan(p_hat, n, alpha)

    print(f"\nInterval Kepercayaan = ({bawah:}, {atas:})")

# Menjalankan program
main()

```

4.2 Source Code R

A. Pemrograman 1 :

Program Menghitung Jumlah Kata dan Jumlah Kalimat

```
# Membaca teks dari pengguna
teks <- readline("Masukkan teks: ")

# Memisahkan teks menjadi kata-kata berdasarkan spasi
kata <- strsplit(teks, " ")[[1]]

# Menghitung jumlah kata
jumlah_kata <- length(kata)

# Menghitung jumlah tanda titik sebagai jumlah kalimat
jumlah_kalimat <- lengths(regmatches(teks, gregexpr("\\.", teks)))

# Menampilkan hasil
cat("\nHasil Perhitungan\n")
cat("Jumlah kata :", jumlah_kata, "\n")
cat("Jumlah kalimat :", jumlah_kalimat)
```

B. Pemrograman 2 :

```
# Program 2 - Membuat objek string K1 sampai K4

# String yang mengandung tanda petik tunggal
K1 = "Saya tak 'kan menyerah."

# String yang mengandung tanda petik ganda
K2 = 'Ia berkata, "Aku menyayangimu."'

# String yang mengandung petik tunggal dan ganda
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""

# String biasa
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

# Menampilkan isi string
print(K1)
print(K2)
print(K3)
print(K4)
```

C. Pemrograman 3 :

Program Mencari Akar Persamaan Kuadrat

```

a <- as.numeric(readline("Masukkan nilai a : "))
b <- as.numeric(readline("Masukkan nilai b : "))
c <- as.numeric(readline("Masukkan nilai c : "))

```

```

D <- b^2 - 4*a*c

```

```

if (D > 0) {

```

```

    x1 <- (-b + sqrt(D)) / (2*a)
    x2 <- (-b - sqrt(D)) / (2*a)

```

```

    cat("Akar real berbeda\n")
    cat(sprintf("x1 = %.3f\n", x1))
    cat(sprintf("x2 = %.3f\n", x2))

```

```

} else if (D == 0) {

```

```

    x <- -b / (2*a)

```

```

    cat("Akar real kembar\n")
    cat(sprintf("x1 = x2 = %.3f\n", x))

```

```

} else {

```

```

    cat("Persamaan memiliki akar-akar imajiner\n")

```

```

}

```

D. Pemrograman 4 :

```

#Program 4

```

```

nip <- readline("Masukkan NIP ASN: ")

```

```

if (nchar(nip) != 18) {
    cat("NIP harus terdiri dari 18 digit\n")
} else {

```

```

    tahun <- substr(nip, 1, 4)
    bulan <- substr(nip, 5, 6)
    tanggal <- substr(nip, 7, 8)

```

```

    bulan_num <- as.numeric(bulan)
    tanggal_num <- as.numeric(tanggal)

```

```

    if (bulan == "01") {

```

```

nama_bulan <- "Januari"
} else if (bulan == "02") {
nama_bulan <- "Februari"
} else if (bulan == "03") {
nama_bulan <- "Maret"
} else if (bulan == "04") {
nama_bulan <- "April"
} else if (bulan == "05") {
nama_bulan <- "Mei"
} else if (bulan == "06") {
nama_bulan <- "Juni"
} else if (bulan == "07") {
nama_bulan <- "Juli"
} else if (bulan == "08") {
nama_bulan <- "Agustus"
} else if (bulan == "09") {
nama_bulan <- "September"
} else if (bulan == "10") {
nama_bulan <- "Oktober"
} else if (bulan == "11") {
nama_bulan <- "November"
} else if (bulan == "12") {
nama_bulan <- "Desember"
} else {
nama_bulan <- "Tidak Valid"
}

if (nama_bulan == "Tidak Valid") {
cat("Bulan pada NIP tidak valid\n")
} else if (tanggal_num < 1 || tanggal_num > 31) {
cat("Tanggal pada NIP tidak valid\n")
} else {
cat("Tanggal lahir ASN:",
tanggal, nama_bulan, tahun)
}
}

```

E. Pemrograman 5 :

```

# Fungsi untuk menghitung jarak Euclidean
hitungJarak <- function(U, P) {
  sqrt(sum((U - P)^2))
}

# Pusat cluster

```

```

A <- c(2, 1, 3)
B <- c(1, -4, 6)
C <- c(-2, 3, -2)

# Input titik U
U <- as.numeric(strsplit(
  readline("Masukkan x1 x2 x3: "),
  " ")
)[[1]])

# Validasi input
if (length(U) != 3 || any(is.na(U))) {
  cat("Input tidak valid! Masukkan tepat 3 angka.")

} else {

  jarakA <- hitungJarak(U, A)
  jarakB <- hitungJarak(U, B)
  jarakC <- hitungJarak(U, C)

  if (jarakA <= jarakB && jarakA <= jarakC) {
    cluster <- "A"
  } else if (jarakB <= jarakA && jarakB <= jarakC) {
    cluster <- "B"
  } else {
    cluster <- "C"
  }

  cat("Jarak ke A =", round(jarakA, 2), "\n")
  cat("Jarak ke B =", round(jarakB, 2), "\n")
  cat("Jarak ke C =", round(jarakC, 2), "\n")
  cat("Titik U berada pada Cluster", cluster)

}

```

F. Pemrograman 6 :

```

# Program 6 : Interval Kepercayaan Proporsi
# Fungsi menentukan nilai z
nilai_z <- function(alpha) {
  if (alpha == 0.10) {
    return(1.645)
  } else if (alpha == 0.05) {
    return(1.96)
  } else {

```

```

    return(NA)
  }
}

# Fungsi menghitung interval kepercayaan
interval_kepercayaan <- function(p_hat, n, alpha) {
  z <- nilai_z(alpha)

  margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)

  batas_bawah <- p_hat - margin_error
  batas_atas <- p_hat + margin_error

  return(c(batas_bawah, batas_atas))
}

# Program utama
main <- function() {
  cat("=== Interval Kepercayaan Proporsi ===\n")
  p_hat <- as.numeric(readline("Masukkan proporsi sampel ( $\hat{p}$ ): "))
  n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
  alpha <- as.numeric(readline("Masukkan alpha (0.10 atau 0.05): "))

  # Validasi proporsi
  if (p_hat < 0 || p_hat > 1) {
    cat("Error: Proporsi harus berada antara 0 dan 1.\n")
    return()
  }

  # Validasi alpha
  if (is.na(nilai_z(alpha))) {
    cat("Error: Alpha harus 0.10 atau 0.05.\n")
    return()
  }

  hasil <- interval_kepercayaan(p_hat, n, alpha)

  cat("\nInterval Kepercayaan = (",
      round(hasil[1], 4), ", ",
      round(hasil[2], 4), ")\n", sep = "")
}

# Menjalankan program
main()

```


BAB V

PENJELASAN PROGRAM

5.1 Penjelasan Line-by-Line-Code R

A. Pemrograman 1 :

Baris Kode	Penjelasan
# Program Menghitung Jumlah Kata dan Jumlah Kalimat	Komentar yang menjelaskan tujuan program, yaitu untuk menghitung jumlah kata dan jumlah kalimat dalam suatu teks.
# Membaca teks dari pengguna teks <- readline("Masukkan teks: ")	Digunakan untuk meminta pengguna memasukkan teks melalui keyboard. Input yang diberikan akan disimpan ke dalam variabel teks.
# Memisahkan teks menjadi kata-kata berdasarkan spasi kata <- strsplit(teks, " ")[[1]]	Memisahkan teks menjadi beberapa kata berdasarkan spasi (" "). Hasil pemisahan disimpan dalam variabel kata.
# Menghitung jumlah kata jumlah_kata <- length(kata)	Menghitung banyaknya elemen pada variabel kata, yang berarti menghitung jumlah kata dalam teks.
# Menghitung jumlah tanda titik sebagai jumlah kalimat jumlah_kalimat <- lengths(regmatches(teks, gregexpr("\\.", teks)))	Menghitung jumlah tanda titik (.) pada teks. Banyaknya tanda titik digunakan sebagai jumlah kalimat.
cat("\nHasil Perhitungan\n")	Menampilkan judul atau teks "Hasil Perhitungan" agar output lebih rapi dan mudah dibaca.
cat("Jumlah kata :", jumlah_kata, "\n")	Menampilkan hasil perhitungan jumlah kata ke layar.
cat("Jumlah kalimat :", jumlah_kalimat)	Menampilkan hasil perhitungan jumlah kalimat ke layar.

B. Pemrograman 2 :

Baris Kode	Penjelasan
# Program 2 - Membuat objek string K1 sampai K4	Komentar yang menjelaskan tujuan program, yaitu membuat dan menampilkan empat objek string.
K1 = "Saya tak 'kan menyerah."	Membuat variabel K1 yang berisi kalimat dengan tanda petik tunggal (').
K2 = 'Ia berkata, "Aku menyayangimu."'	Membuat variabel K2 yang berisi kalimat dengan tanda petik ganda (").
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""	Membuat variabel K3 yang berisi kalimat dengan kombinasi tanda petik tunggal dan tanda petik ganda. Karakter \ digunakan untuk menampilkan tanda petik ganda sebagai bagian dari teks.

K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."	Membuat variabel K4 yang berisi kalimat biasa tanpa karakter khusus.
print(K1)	Menampilkan isi variabel K1 ke layar.
print(K2)	Menampilkan isi variabel K1 ke layar
print(K3)	Menampilkan isi variabel K1 ke layar
print(K4)	Menampilkan isi variabel K1 ke layar

C. Pemrograman 3 :

Baris Kode	Penjelasan
# Program Mencari Akar Persamaan Kuadrat	Komentar yang menjelaskan tujuan program. Tidak dieksekusi oleh R.
a <- as.numeric(readline("Masukkan nilai a : "))	Meminta pengguna memasukkan nilai koefisien a, kemudian mengubah input menjadi tipe numerik dan menyimpannya ke variabel a.
b <- as.numeric(readline("Masukkan nilai b : "))	Meminta pengguna memasukkan nilai koefisien b, lalu menyimpannya ke variabel b
c <- as.numeric(readline("Masukkan nilai c : "))	Meminta pengguna memasukkan nilai koefisien b, lalu menyimpannya ke variabel c
D <- b^2 - 4*a*c	Menghitung diskriminan (D) menggunakan rumus $D = b^2 - 4ac$
if (D > 0) {	Mengecek apakah diskriminan lebih besar dari 0. Jika benar, persamaan memiliki dua akar real yang berbeda.
x1 <- (-b + sqrt(D)) / (2*a)	Menghitung akar pertama (x1) menggunakan rumus kuadrat.
x2 <- (-b - sqrt(D)) / (2*a)	Menghitung akar pertama (x2) menggunakan rumus kuadrat.
cat("Akar real berbeda\n")	Menampilkan pesan bahwa persamaan memiliki dua akar real yang berbeda.
cat(sprintf("x1 = %.3f\n", x1))	Menampilkan nilai x1 dengan 3 angka di belakang koma
cat(sprintf("x2 = %.3f\n", x2))	Menampilkan nilai x2 dengan 3 angka di belakang koma
} else if (D == 0) {	Mengecek apakah diskriminan sama dengan 0. Jika benar, persamaan memiliki akar real kembar.
x <- -b / (2*a)	Menghitung akar kembar menggunakan rumus $x = -b/(2*a)$
cat("Akar real kembar\n")	Menampilkan pesan bahwa persamaan memiliki akar real kembar.
cat(sprintf("x1 = x2 = %.3f\n", x))	Menampilkan nilai akar kembar dengan 3 angka di belakang koma.
} else {	Kondisi yang dijalankan jika diskriminan kurang dari 0.

cat("Persamaan memiliki akar-akar imajiner\n")	Menampilkan pesan bahwa persamaan memiliki akar imajiner (tidak memiliki akar real).
}	Menutup blok percabangan if-else.

D. Pemrograman 4 :

Baris Kode	Penjelasan
nip <- readline("Masukkan NIP ASN: ")	Membaca input NIP ASN dari pengguna dan menyimpannya ke variabel nip
if (nchar(nip) != 18) { ... }	Memeriksa apakah panjang NIP tepat 18 digit. Jika tidak, program menampilkan pesan error.
tahun <- substr(nip, 1, 4)	Mengambil 4 digit pertama NIP sebagai tahun lahir
bulan <- substr(nip, 5, 6)	Mengambil digit ke-5 sampai ke-6 sebagai bulan lahir .
tanggal <- substr(nip, 7, 8)	Mengambil digit ke-7 sampai ke-8 sebagai tanggal lahir
if (bulan == "01") ... else if ... else	Mengubah kode bulan dari NIP menjadi nama bulan, misalnya "01" menjadi Januari , "02" menjadi Februari , dan seterusnya sampai "12" menjadi Desember . Jika tidak sesuai, bulan dianggap tidak valid.
if (nama_bulan == "Tidak Valid") { ... }	Memeriksa apakah bulan hasil ekstraksi valid atau tidak. Jika tidak valid, program menampilkan pesan kesalahan.
`else if (as.numeric(tanggal) < 1	Memeriksa apakah tanggal hasil ekstraksi valid atau tidak. Jika tidak valid, program menampilkan pesan kesalahan.
cat("Tanggal lahir ASN:", tanggal, nama_bulan, tahun)	Menampilkan hasil akhir berupa tanggal lahir ASN dalam format tanggal nama_bulan tahun .

E. Pemrograman 5 :

Baris Kode	Penjelasan
A <- c(2, 1, 3)	Mendefinisikan titik pusat Cluster A dengan koordinat ((2,1,3)).
B <- c(1, -4, 6)	Mendefinisikan titik pusat Cluster B dengan koordinat ((1,-4,6))
C <- c(-2, 3, -2)	Mendefinisikan titik pusat Cluster C dengan koordinat ((-2,3,-2)).
U <- as.numeric(strsplit(readline("Masukkan x1 x2 x3: "), " ")[[1]])	Membaca input titik (U) dari pengguna, memisahkan berdasarkan spasi, lalu mengubahnya menjadi angka agar dapat

	digunakan dalam perhitungan.
<code>if (length(U) != 3 any(is.na(U))) { ... }</code>	Memvalidasi input. Program memeriksa apakah pengguna memasukkan tepat 3 angka dan memastikan tidak ada nilai yang gagal dibaca sebagai angka.
<code>jarakA <- sqrt(sum((U - A)^2))</code>	Menghitung jarak Euclidean antara titik (U) dan pusat Cluster A.
<code>jarakB <- sqrt(sum((U - B)^2))</code>	Menghitung jarak Euclidean antara titik (U) dan pusat Cluster B.
<code>jarakC <- sqrt(sum((U - C)^2))</code>	Menghitung jarak Euclidean antara titik (U) dan pusat Cluster C.
<code>if (jarakA <= jarakB && jarakA <= jarakC) { ... } else if ... else</code>	Membandingkan ketiga jarak. Jika jarak ke A paling kecil, titik dimasukkan ke Cluster A; jika jarak ke B paling kecil, ke Cluster B; selain itu ke Cluster C.
<code>cat("Titik U berada pada Cluster", cluster)</code>	Menampilkan hasil akhir berupa cluster tempat titik (U) berada, yaitu Cluster A, B, atau C.

F. Pemrograman 6:

Baris Kode	Penjelasan
<code># Program 6 : Interval Kepercayaan Proporsi</code>	Komentar yang menunjukkan nama program. Komentar tidak akan dijalankan oleh R.
<code>nilai_z <- function(alpha) {</code>	Mendefinisikan fungsi nilai_z() yang menerima parameter alpha untuk menentukan nilai z.
<code>if (alpha == 0.10) {</code>	Memeriksa apakah nilai alpha sama dengan 0,10.
<code> return(1.645)</code>	Mengembalikan nilai z sebesar 1,645 jika alpha = 0,10.
<code> } else if (alpha == 0.05) {</code>	Jika kondisi sebelumnya tidak terpenuhi, program memeriksa apakah alpha = 0,05.
<code> return(1.96)</code>	Mengembalikan nilai z sebesar 1,96 jika alpha = 0,05.
<code> } else {</code>	Jika alpha bukan 0,10 maupun 0,05 akan diberikan kondisi lain.
<code> return(NA)</code>	Mengembalikan nilai NA (<i>Not Available</i>) sebagai tanda bahwa alpha tidak valid.
<code> }</code>	Menutup fungsi else.
<code>}</code>	Menutup fungsi nilai z.
<code>interval_kepercayaan <- function(p_hat, n, alpha) {</code>	Mendefinisikan fungsi untuk menghitung interval kepercayaan.
<code> z <- nilai_z(alpha)</code>	Memanggil fungsi nilai_z() untuk memperoleh nilai z sesuai alpha.
<code> margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)</code>	Menghitung <i>margin of error</i> menggunakan rumus yang tertera pada soal.

<code>batas_bawah <- p_hat - margin_error</code>	Menghitung batas bawah interval kepercayaan.
<code>batas_atas <- p_hat + margin_error</code>	Menghitung batas atas interval kepercayaan.
<code>return(c(batas_bawah, batas_atas))</code>	Mengembalikan hasil berupa vektor yang berisi batas bawah dan batas atas.
<code>}</code>	Menutup fungsi <code>interval_kepercayaan()</code>
<code>main <- function() {</code>	Mendefinisikan program utama(main) yang menjalankan seluruh program.
<code>cat("=== Interval Kepercayaan Proporsi ===\n")</code>	Menampilkan judul program pada <i>output</i> .
<code>p_hat <- as.numeric(readline("Masukkan proporsi sampel ($\hat{p}$): "))</code>	Meminta pengguna memasukkan proporsi dan mengubah <i>input</i> menjadi numerik.
<code>n <- as.numeric(readline("Masukkan ukuran sampel (n): "))</code>	Meminta pengguna memasukkan ukuran sampel dan mengubah <i>input</i> menjadi numerik.
<code>alpha <- as.numeric(readline("Masukkan alpha (0.10 atau 0.05): "))</code>	Meminta pengguna memasukkan nilai alpha dan mengubah <i>input</i> menjadi numerik.
<code>if (p_hat < 0 p_hat > 1) {</code>	Memvalidasi nilai proporsi (<code>p_hat</code>). Program memeriksa apakah nilai proporsi yang dimasukkan berada di luar rentang yang diperbolehkan, yaitu 0 sampai 1.
<code>cat("Error: Proporsi harus berada antara 0 dan 1.\n")</code>	Menampilkan pesan kesalahan jika proporsi tidak valid.
<code>return()</code>	Menghentikan program jika proporsi tidak valid.
<code>}</code>	Menutup blok validasi proporsi.
<code>if (is.na(nilai_z(alpha))) {</code>	Memeriksa apakah fungsi <code>nilai_z()</code> menghasilkan NA, yang berarti alpha tidak valid.
<code>cat("Error: Alpha harus 0.10 atau 0.05.\n")</code>	Menampilkan pesan kesalahan jika alpha tidak sesuai ketentuan.
<code>return()</code>	Menghentikan program jika alpha tidak valid.
<code>}</code>	Menutup blok validasi alpha.
<code>hasil <- interval_kepercayaan(p_hat, n, alpha)</code>	Memanggil fungsi <code>interval_kepercayaan()</code> untuk menghitung batas bawah dan batas atas interval kepercayaan, kemudian menyimpannya ke variabel <code>hasil</code> .
<code>cat("\nInterval Kepercayaan = (",</code>	Mulai menampilkan hasil interval kepercayaan. <code>\n</code> digunakan untuk membuat baris baru.
<code>round(hasil[1], 4), ", ",</code>	Menampilkan batas bawah interval yang dibulatkan menjadi 4 angka di belakang koma.
<code>round(hasil[2], 4), ")\n", sep = "")</code>	Menampilkan batas atas interval yang

	dibulatkan menjadi 4 angka di belakang koma, lalu menutup tampilan interval. <code>sep = ""</code> digunakan agar tidak ada spasi tambahan antar bagian output.
}	Menutup fungsi main()
main()	Memanggil fungsi main() sehingga seluruh program dijalankan.

5.2 Penjelasan Line-by-Line-Code Python

A. Pemrograman 1 :

Baris Kode	Penjelasan
# Soal Program 1	Komentar yang menunjukkan bahwa kode ini merupakan penyelesaian untuk Soal Program 1.
# Program Menghitung Jumlah Kata dan Jumlah Kalimat	Komentar yang menjelaskan tujuan program, yaitu menghitung jumlah kata dan jumlah kalimat dalam sebuah teks.
teks = input("Masukkan teks: ")	Digunakan untuk meminta pengguna memasukkan teks melalui keyboard. Input yang diberikan akan disimpan ke dalam variabel <code>teks</code> .
kata = teks.split()	Memisahkan teks menjadi beberapa kata berdasarkan spasi. Hasil pemisahan disimpan dalam variabel <code>kata</code> berupa list.
jumlah_kata = len(kata)	Menghitung jumlah kata dengan menghitung banyaknya elemen yang terdapat pada list kata.
jumlah_kalimat = teks.count(".")	Menghitung jumlah tanda titik (.) yang terdapat pada teks. Banyaknya tanda titik dianggap sebagai jumlah kalimat.
print("\nHasil Perhitungan")	Menampilkan judul "Hasil perhitungan" agar output lebih rapi dan mudah dibaca
print("Jumlah kata :", jumlah_kata)	Menampilkan hasil perhitungan jumlah kata ke layar.
print("Jumlah kalimat :", jumlah_kalimat)	Menampilkan hasil perhitungan jumlah kalimat ke layar.

B. Pemrograman 2 :

Baris Kode	Penjelasan
# Soal Program 2	Komentar yang menunjukkan bahwa kode ini merupakan penyelesaian untuk Soal Program 2.
# Program 2 - Membuat objek string K1 sampai K4	Komentar yang menjelaskan tujuan program, yaitu membuat dan menampilkan empat objek string.
K1 = "Saya tak 'kan menyerah."	Membuat variabel K1 yang berisi kalimat "Saya tak 'kan menyerah.". Kalimat ini

	mengandung tanda petik tunggal (') sebagai bagian dari teks.
K2 = 'Ia berkata, "Aku menyayangimu."'	Membuat variabel K2 yang berisi kalimat dengan tanda petik ganda ("). Penggunaan petik tunggal di luar string memungkinkan petik ganda ditampilkan sebagai bagian dari teks.
K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""	Membuat variabel K3 yang berisi kalimat dengan kombinasi tanda petik tunggal dan tanda petik ganda. Karakter \ digunakan sebagai <i>escape character</i> agar tanda petik ganda dapat ditampilkan sebagai isi string.
K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."	Membuat variabel K4 yang berisi kalimat biasa tanpa karakter khusus.
print(K1)	Menampilkan isi variabel K1 ke layar.
print(K2)	Menampilkan isi variabel K2 ke layar.
print(K3)	Menampilkan isi variabel K3 ke layar.
print(K4)	Menampilkan isi variabel K4 ke layar.

C. Pemrograman 3 :

Baris Kode	Penjelasan
# Program Mencari Akar Persamaan Kuadrat	Komentar yang menjelaskan tujuan program. Baris ini tidak dieksekusi oleh Python.
a = float(input("Masukkan nilai a : "))	Meminta pengguna memasukkan nilai koefisien a. Fungsi input() menerima input sebagai teks, kemudian float() mengubahnya menjadi bilangan desimal (float). Nilai disimpan ke variabel a
b = float(input("Masukkan nilai b : "))	Meminta pengguna memasukkan nilai koefisien b. Fungsi input() menerima input sebagai teks, kemudian float() mengubahnya menjadi bilangan desimal (float). Nilai disimpan ke variabel b
c = float(input("Masukkan nilai c : "))	Meminta pengguna memasukkan nilai koefisien b. Fungsi input() menerima input sebagai teks, kemudian float() mengubahnya menjadi bilangan desimal (float). Nilai disimpan ke variabel b
D = b**2 - 4*a*c	Menghitung nilai diskriminan (D) menggunakan rumus $D = b^2 - 4ac$. Operator ** digunakan untuk perpangkatan.
if D > 0:	Memeriksa apakah nilai diskriminan lebih besar dari 0. Jika benar, persamaan memiliki dua akar real yang berbeda.
x1 = (-b + (D**0.5)) / (2*a)	Menghitung akar pertama (x1) menggunakan rumus kuadrat. Ekspresi

	D**0.5 digunakan untuk menghitung akar kuadrat dari D.
$x2 = (-b - (D^{**0.5})) / (2*a)$	Menghitung akar pertama (x2) menggunakan rumus kuadrat. Ekspresi D**0.5 digunakan untuk menghitung akar kuadrat dari D.
print("Akar real berbeda")	Menampilkan pesan bahwa persamaan memiliki dua akar real yang berbeda.
print("x1 = %.3f" % x1)	Menampilkan nilai akar pertama (x1) dengan format 3 angka di belakang koma.
print("x2 = %.3f" % x2)	Menampilkan nilai akar pertama (x2) dengan format 3 angka di belakang koma.
elif D == 0:	Memeriksa apakah diskriminan sama dengan 0. Jika benar, persamaan memiliki akar real kembar.
$x = -b / (2*a)$	Menghitung nilai akar kembar
print("Akar real kembar")	Menampilkan pesan bahwa persamaan memiliki akar real kembar.
print("x1 = x2 = %.3f" % x)	Menampilkan nilai akar kembar dengan format 3 angka di belakang koma.
else:	Kondisi yang dijalankan jika semua kondisi sebelumnya salah, yaitu ketika diskriminan kurang dari 0.
print("Persamaan memiliki akar-akar imajiner")	Menampilkan pesan bahwa persamaan memiliki akar imajiner (kompleks), sehingga tidak memiliki akar real.

D. Pemrograman 4 :

Baris Kode	Penjelasan
nip = input("Masukkan NIP ASN: ")	Membaca input NIP ASN dari pengguna dan menyimpannya ke variabel nip
if len(nip) != 18:	Memeriksa apakah panjang NIP tepat 18 digit. Jika tidak, program menampilkan pesan error.
print("NIP harus terdiri dari 18 digit")	Menampilkan pesan kesalahan jika panjang NIP tidak sesuai ketentuan.
else:	Jika panjang NIP valid, program melanjutkan proses pengambilan data tanggal lahir dari NIP
tahun = nip[0:4]	Mengambil 4 digit pertama NIP sebagai tahun lahir
bulan = nip[4:6]	Mengambil digit ke-5 sampai ke-6 sebagai bulan lahir .
tanggal = nip[6:8]	Mengambil digit ke-7 sampai ke-8 sebagai tanggal lahir
if bulan == "01": ... elif... else	Mengubah kode bulan dari NIP menjadi nama bulan, misalnya "01" menjadi Januari , "02" menjadi Februari , dan

	seterusnya sampai "12" menjadi Desember . Jika tidak sesuai, bulan dianggap tidak valid.
if nama_bulan == "Tidak Valid" :	Memeriksa apakah bulan hasil ekstraksi valid atau tidak. Jika tidak valid, program menampilkan pesan kesalahan.
elif int(tanggal) < 1 or int(tanggal) > 31 :	Memeriksa apakah tanggal hasil ekstraksi valid atau tidak. Jika tidak valid, program menampilkan pesan kesalahan.
print("Tanggal lahir ASN:", tanggal, nama_bulan, tahun)	Menampilkan hasil akhir berupa tanggal lahir ASN dalam format tanggal nama_bulan tahun .

E. Pemrograman 5 :

Baris Kode	Penjelasan
#Soal nomor 5	Komentar yang menunjukkan bahwa program ini merupakan penyelesaian soal nomor 5.
import math	Mengimpor library math agar dapat menggunakan fungsi matematika, seperti sqrt() untuk menghitung akar kuadrat.
def hitung_jarak(titik1, titik2):	Mendefinisikan fungsi hitung_jarak () yang menerima dua parameter berupa titik koordinat.
return math.sqrt(	Mengembalikan hasil perhitungan akar kuadrat dari jumlah kuadrat selisih koordinat kedua titik.
(titik1[0] - titik2[0]) ** 2 +	Menghitung kuadrat selisih koordinat pertama (sumbu x).
(titik1[1] - titik2[1]) ** 2 +	Menghitung kuadrat selisih koordinat kedua (sumbu y).
(titik1[2] - titik2[2]) ** 2	Menghitung kuadrat selisih koordinat ketiga (sumbu z).
)	Menutup fungsi math.sqrt().
def tentukan_cluster(titik_u):	Mendefinisikan fungsi tentukan_cluster() untuk menentukan cluster dari titik U.
cluster_a = (2, 1, 3)	Menyimpan koordinat pusat cluster A.
cluster_b = (1, -4, 6)	Menyimpan koordinat pusat cluster B.
cluster_c = (-2, 3, -2)	Menyimpan koordinat pusat cluster C.
jarak_a = hitung_jarak(titik_u, cluster_a)	Menghitung jarak titik U ke cluster A.
jarak_b = hitung_jarak(titik_u, cluster_b)	Menghitung jarak titik U ke cluster B.
jarak_c = hitung_jarak(titik_u, cluster_c)	Menghitung jarak titik U ke cluster C.
print(f'Jarak ke Cluster A = {jarak_a:}')"	Menampilkan hasil jarak titik U ke cluster A.
print(f'Jarak ke Cluster B = {jarak_b:}')"	Menampilkan hasil jarak titik U ke cluster B.
print(f'Jarak ke Cluster C = {jarak_c:}')"	Menampilkan hasil jarak titik U ke cluster C.

if jarak_a <= jarak_b and jarak_a <= jarak_c:	Memeriksa apakah jarak ke cluster A merupakan jarak yang paling kecil atau sama dengan jarak terkecil lainnya.
return "A"	Mengembalikan nilai "A" jika titik termasuk cluster A.
elif jarak_b <= jarak_a and jarak_b <= jarak_c:	Jika bukan cluster A, program memeriksa apakah jarak ke cluster B yang paling kecil atau sama dengan jarak terkecil lainnya.
return "B"	Mengembalikan nilai "B" jika titik termasuk cluster B.
else:	Jika bukan cluster A maupun B, maka dianggap paling dekat dengan cluster C.
return "C"	Mengembalikan nilai "C" jika titik termasuk cluster C.
try:	Memulai blok <i>exception handling</i> untuk menangani kemungkinan kesalahan saat <i>input</i> .
x1 = float(input("Masukkan x1: "))	Meminta pengguna memasukkan koordinat pertama dan mengubahnya menjadi tipe data float.
x2 = float(input("Masukkan x2: "))	Meminta pengguna memasukkan koordinat kedua.
x3 = float(input("Masukkan x3: "))	Meminta pengguna memasukkan koordinat ketiga.
titik_u = (x1, x2, x3)	Menggabungkan ketiga koordinat menjadi sebuah tuple bernama titik_u
hasil = tentukan_cluster(titik_u)	Memanggil fungsi tentukan_cluster() untuk menentukan cluster titik U dan menyimpan hasilnya ke variabel hasil.
print("Titik U termasuk Cluster", hasil)	Menampilkan hasil klasifikasi cluster kepada pengguna.
except ValueError:	Menangkap kesalahan jika pengguna memasukkan data yang bukan berupa angka.
print("Input harus berupa angka!")	Menampilkan pesan kesalahan apabila input tidak dapat dikonversi menjadi tipe data numerik.

F. Pemrograman 6 :

Baris Kode	Penjelasan
import math	Mengimpor library math agar dapat menggunakan fungsi matematika, seperti sqrt() untuk menghitung akar kuadrat.
def nilai_z(alpha):	Mendefinisikan fungsi nilai_z() yang menerima parameter alpha untuk menentukan nilai z.
if alpha == 0.10:	Memeriksa apakah nilai alpha sama dengan 0,10.

return 1.645	Mengembalikan nilai z sebesar 1,645 jika $\alpha = 0,10$.
elif $\alpha == 0.05$:	Jika kondisi sebelumnya tidak terpenuhi, program memeriksa apakah $\alpha = 0,05$.
return 1.96	Mengembalikan nilai z sebesar 1,96 jika $\alpha = 0,05$.
else:	Jika nilai α bukan 0,10 maupun 0,05 akan diberikan kondisi lain.
return None	Mengembalikan nilai None sebagai tanda bahwa α tidak valid.
def interval_kepercayaan(p_hat, n, alpha):	Mendefinisikan fungsi untuk menghitung interval kepercayaan proporsi.
z = nilai_z(alpha)	Memanggil fungsi nilai_z() untuk mendapatkan nilai z sesuai α .
margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)	Menghitung <i>margin of error</i> menggunakan rumus yang tertera.
batas_bawah = p_hat - margin_error	Menghitung batas bawah interval kepercayaan.
batas_atas = p_hat + margin_error	Menghitung batas atas interval kepercayaan.
return batas_bawah, batas_atas	Mengembalikan dua nilai, yaitu batas bawah dan batas atas interval kepercayaan.
def main():	Mendefinisikan fungsi utama (main) yang akan menjalankan seluruh program.
print("=== Interval Kepercayaan Proporsi ===")	Menampilkan judul program pada layar.
p_hat = float(input("Masukkan proporsi sampel ($\hat{p}$): "))	Meminta pengguna memasukkan nilai proporsi sampel dan mengubahnya menjadi tipe data float.
n = int(input("Masukkan ukuran sampel (n): "))	Meminta pengguna memasukkan ukuran sampel dan mengubahnya menjadi tipe data float.
alpha = float(input("Masukkan alpha (0.10 atau 0.05): "))	Meminta pengguna memasukkan nilai α dan mengubahnya menjadi tipe data float.
if p_hat < 0 or p_hat > 1:	Memeriksa apakah proporsi berada di luar rentang 0 sampai 1. Operator or berarti salah satu kondisi benar sudah cukup untuk menjalankan blok if.
print("Error: Proporsi harus berada antara 0 dan 1.")	Menampilkan pesan kesalahan jika proporsi tidak valid.
return	Menghentikan eksekusi fungsi main() sehingga program tidak melanjutkan perhitungan.
if nilai_z(alpha) is None:	Memeriksa apakah fungsi nilai_z() mengembalikan None, yang berarti nilai α tidak valid.
print("Error: Alpha harus 0.10 atau 0.05.")	Menampilkan pesan kesalahan jika α

	bukan 0,10 atau 0,05.
return	Menghentikan program jika alpha tidak valid.
bawah, atas = interval_kepercayaan(p_hat, n, alpha)	Memanggil fungsi interval_kepercayaan() untuk menghitung batas bawah dan batas atas, kemudian menyimpannya ke variabel bawah dan atas
print(f"\nInterval Kepercayaan = ({bawah:}, {atas:})")	Menampilkan hasil interval kepercayaan yang terdiri dari batas bawah dan batas atas. \n digunakan untuk membuat baris baru sebelum hasil ditampilkan.
main()	Memanggil fungsi main() sehingga seluruh program dijalankan.

BAB VI PENGUJIAN PROGRAM

6.1 Tabel Pengujian

A. Pemograman 1 :

Skenario Uji	Input	Output	Status
Masukan 1 kalimat	Kelompok kami sedang mengerjakan tugas algoritma pemrograman.	Jumlah kata : 7 Jumlah Kalimat :1	Berhasil
Masukan Beberapa Kalimat	Kami suka dengan tugas besar kali ini karena ilmu yang dipelajari dalam setiap soal sangat bermanfaat untuk kedepannya. Apalagi di era yang sudah canggih ini kita perlu untuk terus belajar teknologi agar tidak ketinggalan. Selain itu video demo yang kami buat semoga dapat bermanfaat bagi orang yang menonton. Terimakasih sekian dari kelompok kami.	Jumlah kata : 52 Jumlah kalimat : 4	Berhasil
Tidak memasukan	-	Jumlah kata : 0 Jumlah kalimat : 0	Berhasil

B. Pemrograman 2 :

Skenario Uji	Output yang diharapkan	Output Program	Status
Menampilkan string K1, K2, K3, K4	Seluruh string ditampilkan sesuai dengan format yang ditentukan pada soal.	Seluruh string berhasil ditampilkan sesuai soal.	Berhasil

C. Pemrograman 3 :

Skenario Uji	Output yang diharapkan	Output Program	Status
$D > 0$ (Input $a = 1$, $b = -5$, $c = 6$)	Menghasilkan akar-akar yang memiliki nilai berbeda	Akar real berbeda : $x_1 : 3.000$ $x_2 : 2.000$	Berhasil
$D = 0$ (Input $a = 1$, $b = -4$, $c = 4$)	Menghasilkan akar kembar yang memiliki nilai sama	Akar real kembar : $x_1 = x_2 = 2.000$	Berhasil
$D < 0$ (input $a = 1$, $b = -2$, $c = 4$)	Menghasilkan akar yang bersifat imajiner	Persamaan ini memiliki akar-akar imajiner	Berhasil

D. Pemrograman 4 :

Skenario Uji	Input	Output Program	Status
NIP valid	1990021720190310	Tanggal lahir ASN: 17 Februari 1990	Berhasil
Panjang NIP kurang dari 18 digit	19900217201903	NIP harus terdiri dari 18 digit	Berhasil
Bulan pada NIP tidak valid	1990171720190310	Bulan pada NIP tidak valid	Berhasil
Tanggal pada NIP tidak valid	1990024220190310	Tanggal pada NIP tidak valid	Berhasil

E. Pemrograman 5 :

Skenario Uji	Input	Output Program	Status
Titik berada pada Cluster A	2 1 3 (R Studio) 3 2 4 (Python)	Titik U berada pada Cluster A	Berhasil
Titik berada pada Cluster B	1 -3 5 (R Studio) -3 5 -1 (Python)	Titik U berada pada Cluster B	Berhasil
Titik berada pada Cluster C	1-1 3 -1	Titik U berada pada Cluster C	Berhasil
Input kurang dari 3 angka	2 1	Input tidak valid! Masukkan tepat 3 angka.	Berhasil
Input lebih dari 3 angka	2 1 3 4	Input tidak valid! Masukkan tepat 3 angka.	Berhasil
Input mengandung huruf	2 a 3	Input tidak valid! Masukkan tepat 3 angka.	Berhasil

F. Pemrograman 6 :

Skenario Uji	Input	Output Program	Status
Menghitung interval kepercayaan dengan $\alpha=0.05$	$\hat{p} = 0,6$, $n = 100$, $\alpha = 0,05$	Interval Kepercayaan = (0.504, 0.696)	Berhasil
Menghitung interval kepercayaan dengan $\alpha=0.10$	$\hat{p} = 0,8$, $n = 30$, $\alpha = 0,10$	Interval Kepercayaan = (0.6799, 0.9201)	Berhasil
Menghitung interval dengan proporsi tidak valid	$\hat{p} = 2$, $n = 30$, $\alpha = 0,05$	Error: Proporsi harus berada antara 0 dan 1. NULL	Berhasil

Menghitung interval dengan alpha tidak valid	$\hat{p} = 0,5$, $n = 100$, $\alpha = 0,30$	Error: Alpha harus 0.10 atau 0.05. NULL	Berhasil
--	---	---	----------

6.2 Screenshot Hasil Running

A. Pemograman 1 :

- Running R

```

1 # Program Menghitung Jumlah Kata dan Jumlah Kalimat
2
3 # Membaca teks dari pengguna
4 teks <- readline("Masukkan teks: ")
5
6 # Memisahkan teks menjadi kata-kata berdasarkan spasi
7 kata <- strsplit(teks, " ")[[1]]
8
9 # Menghitung jumlah kata
10 jumlah_kata <- length(kata)
11
12 # Menghitung jumlah tanda titik sebagai jumlah kalimat
13 jumlah_kalimat <- length(regmatches(teks, gregexpr("\\.", teks)))
14
15 # Menampilkan hasil
16 cat("\nHasil Perhitungan\n")
17 cat("Jumlah kata :", jumlah_kata, "\n")
18 cat("Jumlah kalimat :", jumlah_kalimat, "\n")
19
20 # Test
21 
```

Console Output:

```

> # Menampilkan hasil
> cat("\nHasil Perhitungan\n")
Hasil Perhitungan
> cat("Jumlah kata :", jumlah_kata, "\n")
Jumlah kata : 7
> cat("Jumlah kalimat :", jumlah_kalimat, "\n")
Jumlah kalimat : 1
>

```

Gambar 1. Kondisi Pertama hanya satu kalimat

```

2
3 # Membaca teks dari pengguna
4 teks <- readline("Masukkan teks: ")
5
6 # Memisahkan teks menjadi kata-kata berdasarkan spasi
7 kata <- strsplit(teks, " ")[[1]]
8
9 # Menghitung jumlah kata
10 jumlah_kata <- length(kata)
11
12 # Menghitung jumlah tanda titik sebagai jumlah kalimat
13 jumlah_kalimat <- length(regmatches(teks, gregexpr("\\.", teks)))
14
15 # Menampilkan hasil
16 cat("\nHasil Perhitungan\n")
17 cat("Jumlah kata :", jumlah_kata, "\n")
18 cat("Jumlah kalimat :", jumlah_kalimat, "\n")
19
20 # Test
21 
```

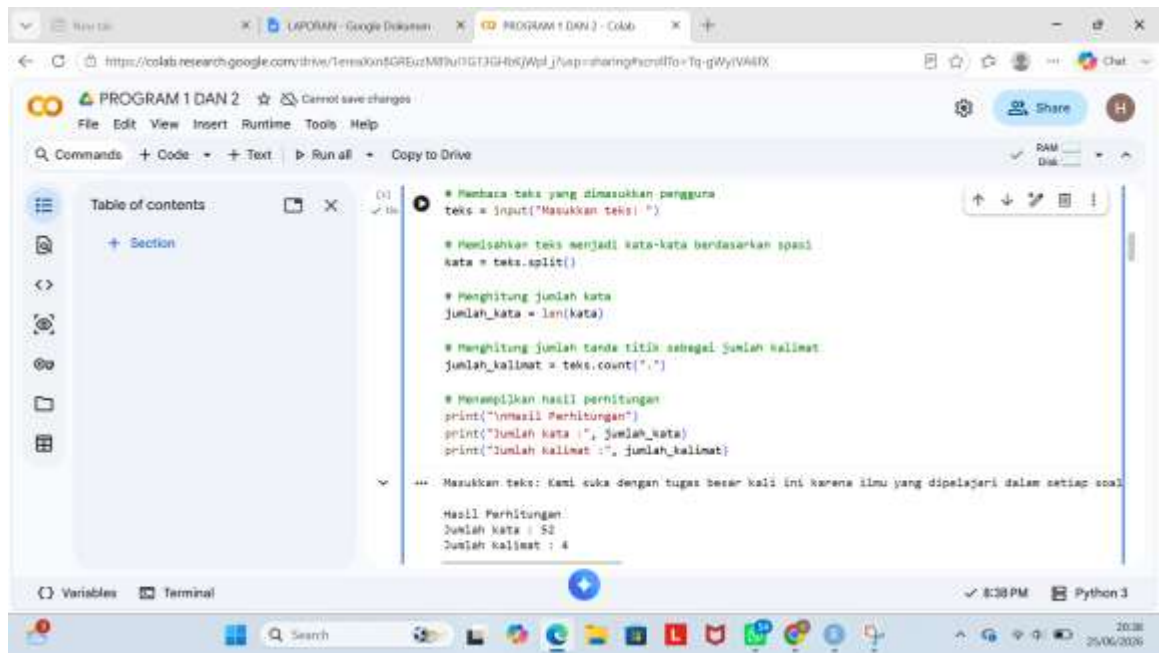
Console Output:

```

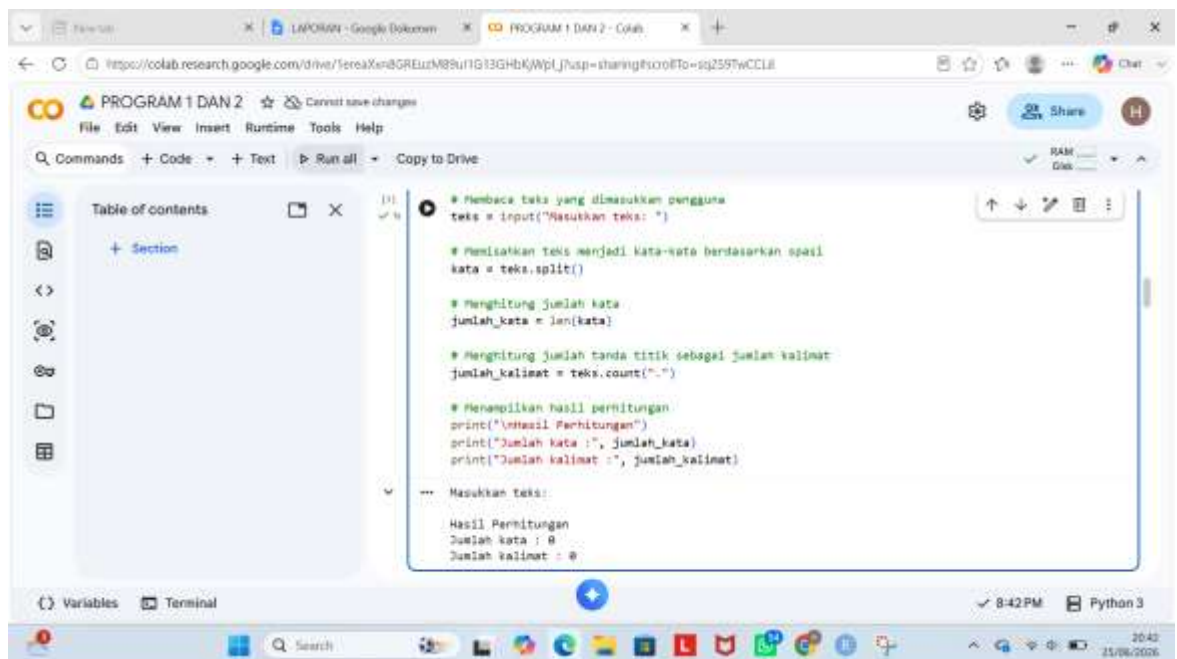
> # Menampilkan hasil
> cat("\nHasil Perhitungan\n")
Hasil Perhitungan
> cat("Jumlah kata :", jumlah_kata, "\n")
Jumlah kata : 52
> cat("Jumlah kalimat :", jumlah_kalimat, "\n")
Jumlah kalimat : 4
>

```

Gambar 2. Kondisi kedua beberapa kalimat



Gambar 2. Kondisi kedua beberapa kalimat



Gambar 3. Kondisi ketiga saat tidak memasukan apapun

B. Pemrograman 2 :

- Running R

The screenshot shows the RStudio interface. The main editor contains the following R code:

```

1 # Program 1 - Membuat objek string K1 sampai K4
2
3 # String yang mengandung tanda petik tunggal
4 K1 = "Saya tak 'kan menyerah.'"
5
6 # String yang mengandung tanda petik ganda
7 K2 = "Ia berkata, 'Aku menyayangimu.'"
8
9 # String yang mengandung petik tunggal dan ganda
10 K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
11
12 # String biasa
13 K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."
14
15 # Menampilkan isi string
16 print(K1)
17 print(K2)
18 print(K3)
19 print(K4)
20
21

```

The console shows the output of the code:

```

> print(K1)
[1] "Saya tak 'kan menyerah.'"
> print(K2)
[1] "Ia berkata, 'Aku menyayangimu.'"
> print(K3)
[1] "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
> print(K4)
[1] "Surat keputusan itu bernomor 62/UN.34/19/2023."
>

```

The Environment pane on the right shows the variables K1, K2, K3, and K4 with their respective string values.

- Running Python

The screenshot shows the Google Colab interface. The main editor contains the following Python code:

```

1 # String yang mengandung tanda petik tunggal
2 K1 = "Saya tak 'kan menyerah.'"
3
4 # String yang mengandung tanda petik ganda
5 K2 = "Ia berkata, 'Aku menyayangimu.'"
6
7 # String yang mengandung petik tunggal dan ganda
8 K3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
9
10 # String biasa
11 K4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."
12
13 # Menampilkan isi string
14 print(K1)
15 print(K2)
16 print(K3)
17 print(K4)
18

```

The output of the code is displayed below the code cell:

```

Saya tak 'kan menyerah.
Ia berkata, 'Aku menyayangimu.
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!
Surat keputusan itu bernomor 62/UN.34/19/2023.

```

C. Pemrograman 3 :

- Running R

```

1 # Program mencari akar persamaan kuadrat
2 a <- as.numeric(readline("masukkan nilai a : "))
3 b <- as.numeric(readline("masukkan nilai b : "))
4 c <- as.numeric(readline("masukkan nilai c : "))
5
6 D <- b^2 - 4*a*c
7
8 if (D > 0) {
9   x1 <- (-b + sqrt(D)) / (2*a)
10  x2 <- (-b - sqrt(D)) / (2*a)
11  cat("Akar real berbeda\n")
12  cat(sprintf("x1 = %.3f\n", x1))
13  cat(sprintf("x2 = %.3f\n", x2))
14 } else if (D == 0) {
15   x <- -b / (2*a)
16   cat("Akar real kembar\n")
17   cat(sprintf("x1 = x2 = %.3f\n", x))
18 } else {
19   cat("Persamaan memiliki akar-akar imajiner\n")
20 }
21
22 (No value)

```

Console Terminal Background Jobs

```

R> # R 4.0.3 - v
+ cat(sprintf("x1 = %.3f\n", x1))
+ cat(sprintf("x2 = %.3f\n", x2))
+ } else if (D == 0) {
+   x <- -b / (2*a)
+   cat("Akar real kembar\n")
+   cat(sprintf("x1 = x2 = %.3f\n", x))
+ } else {
+   cat("Persamaan memiliki akar-akar imajiner\n")
+ }
+
Akar real berbeda
x1 = 3.000
x2 = 2.000
=

```

Gambar 1. Kondisi saat $D > 0$ ($a = 1$, $b = -5$, $c = 6$)

```

1 # Program mencari akar persamaan kuadrat
2 a <- as.numeric(readline("masukkan nilai a : "))
3 b <- as.numeric(readline("masukkan nilai b : "))
4 c <- as.numeric(readline("masukkan nilai c : "))
5
6 D <- b^2 - 4*a*c
7
8 if (D > 0) {
9   x1 <- (-b + sqrt(D)) / (2*a)
10  x2 <- (-b - sqrt(D)) / (2*a)
11  cat("Akar real berbeda\n")
12  cat(sprintf("x1 = %.3f\n", x1))
13  cat(sprintf("x2 = %.3f\n", x2))
14 } else if (D == 0) {
15   x <- -b / (2*a)
16   cat("Akar real kembar\n")
17   cat(sprintf("x1 = x2 = %.3f\n", x))
18 } else {
19   cat("Persamaan memiliki akar-akar imajiner\n")
20 }
21
22 (No value)

```

Console Terminal Background Jobs

```

R> # R 4.0.3 - v
+ cat(sprintf("x1 = %.3f\n", x1))
+ cat(sprintf("x2 = %.3f\n", x2))
+ } else if (D == 0) {
+   x <- -b / (2*a)
+   cat("Akar real kembar\n")
+   cat(sprintf("x1 = x2 = %.3f\n", x))
+ } else {
+   cat("Persamaan memiliki akar-akar imajiner\n")
+ }
+
Akar real kembar
x1 = x2 = 2.000
=

```

Gambar 2. Kondisi saat $D = 0$ ($a = 1$, $b = -4$, $c = 4$)

```

1 # Program Mencari akar persamaan kuadrat
2 a <- as.numeric(readline("Masukkan nilai a : "))
3 b <- as.numeric(readline("Masukkan nilai b : "))
4 c <- as.numeric(readline("Masukkan nilai c : "))
5
6 D <- b^2 - 4*a*c
7
8 if (D > 0) {
9   x1 <- (-b + sqrt(D)) / (2*a)
10  x2 <- (-b - sqrt(D)) / (2*a)
11  cat("Akar real berbeda\n")
12  cat(sprintf("x1 = %.3f\n", x1))
13  cat(sprintf("x2 = %.3f\n", x2))
14 } else if (D == 0) {
15   x <- -b / (2*a)
16   cat("Akar real kembar\n")
17   cat(sprintf("x1 = x2 = %.3f\n", x))
18 } else {
19   cat("Persamaan memiliki akar-akar imajiner\n")
20 }

```

```

> RASI ->
+ cat("Akar real berbeda\n")
+ cat(sprintf("x1 = %.3f\n", x1))
+ cat(sprintf("x2 = %.3f\n", x2))
+ } else if (D == 0) {
+   x <- -b / (2*a)
+   cat("Akar real kembar\n")
+   cat(sprintf("x1 = x2 = %.3f\n", x))
+ } else {
+   cat("Persamaan memiliki akar-akar imajiner\n")
+ }
Persamaan memiliki akar-akar imajiner
>

```

Gambar 3. Kondisi saat $D < 0$ ($a = 1$, $b = -2$, $c = 4$)

- Running Phyton

```

1 # Program Mencari akar persamaan kuadrat
2 a = float(input("Masukkan nilai a : "))
3 b = float(input("Masukkan nilai b : "))
4 c = float(input("Masukkan nilai c : "))
5
6 D = b**2 - 4*a*c
7
8 if D > 0:
9   x1 = (-b + (D**0.5)) / (2*a)
10  x2 = (-b - (D**0.5)) / (2*a)
11  print("Akar real berbeda")
12  print("x1 = %.3f" % x1)
13  print("x2 = %.3f" % x2)
14
15 elif D == 0:
16   x = -b / (2*a)
17   print("Akar real kembar")
18   print("x1 = x2 = %.3f" % x)
19
20 else:
21   print("Persamaan memiliki akar-akar imajiner")

```

```

Masukkan nilai a : 1
Masukkan nilai b : -5
Masukkan nilai c : 6
Akar real berbeda
x1 = 2.000
x2 = -3.000

```

Gambar 1. Kondisi Saat $D > 0$ ($a = 1$, $b = -5$, $c = 6$)

```

# Program Mencari Akar Persamaan Kuadrat
a = float(input("Masukkan nilai a : "))
b = float(input("Masukkan nilai b : "))
c = float(input("Masukkan nilai c : "))

D = b**2 - 4*a*c

if D > 0:
    x1 = (-b + (D**0.5)) / (2*a)
    x2 = (-b - (D**0.5)) / (2*a)
    print("Akar real berbeda")
    print("x1 = %.3f" % x1)
    print("x2 = %.3f" % x2)

elif D == 0:
    x = -b / (2*a)
    print("Akar real kembar")
    print("x1 = x2 = %.3f" % x)

else:
    print("Persamaan memiliki akar-akar imajiner")

```

Masukkan nilai a : 1
 Masukkan nilai b : -4
 Masukkan nilai c : 4
 Akar real kembar
 x1 = x2 = 2.000

Gambar 2. Kondisi saat $D = 0$ ($a = 1, b = -4, c = 4$)

```

# Program Mencari Akar Persamaan Kuadrat
a = float(input("Masukkan nilai a : "))
b = float(input("Masukkan nilai b : "))
c = float(input("Masukkan nilai c : "))

D = b**2 - 4*a*c

if D > 0:
    x1 = (-b + (D**0.5)) / (2*a)
    x2 = (-b - (D**0.5)) / (2*a)
    print("Akar real berbeda")
    print("x1 = %.3f" % x1)
    print("x2 = %.3f" % x2)

elif D == 0:
    x = -b / (2*a)
    print("Akar real kembar")
    print("x1 = x2 = %.3f" % x)

else:
    print("Persamaan memiliki akar-akar imajiner")

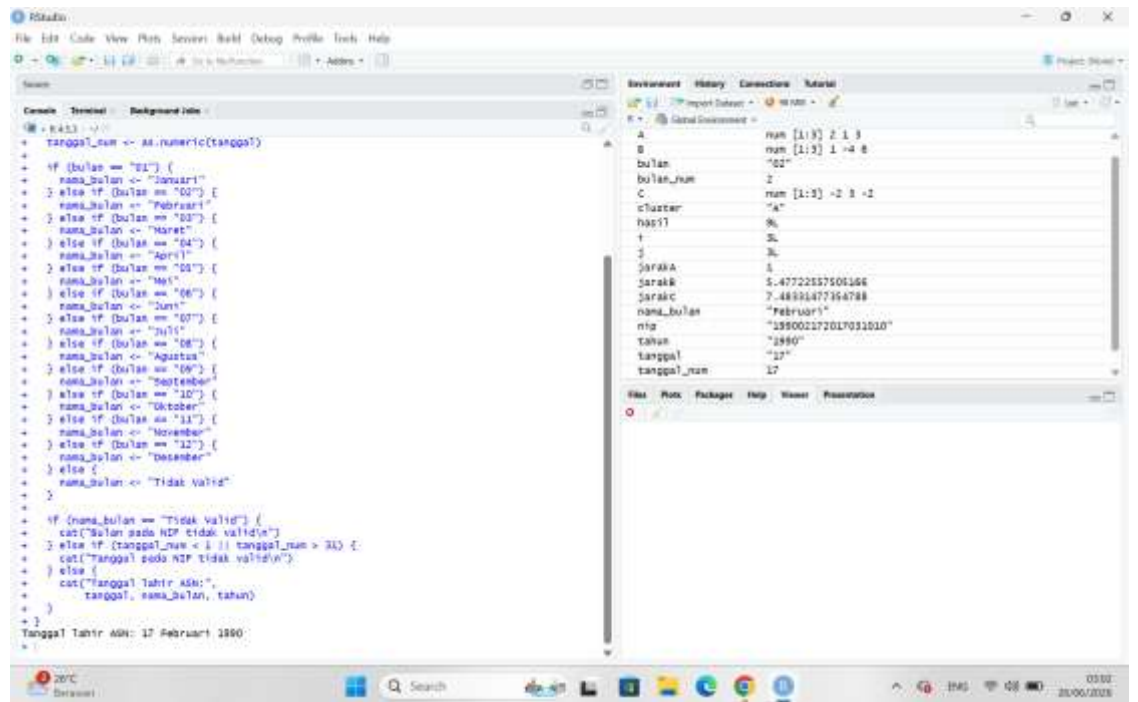
```

Masukkan nilai a : 1
 Masukkan nilai b : -2
 Masukkan nilai c : 4
 Persamaan memiliki akar-akar imajiner

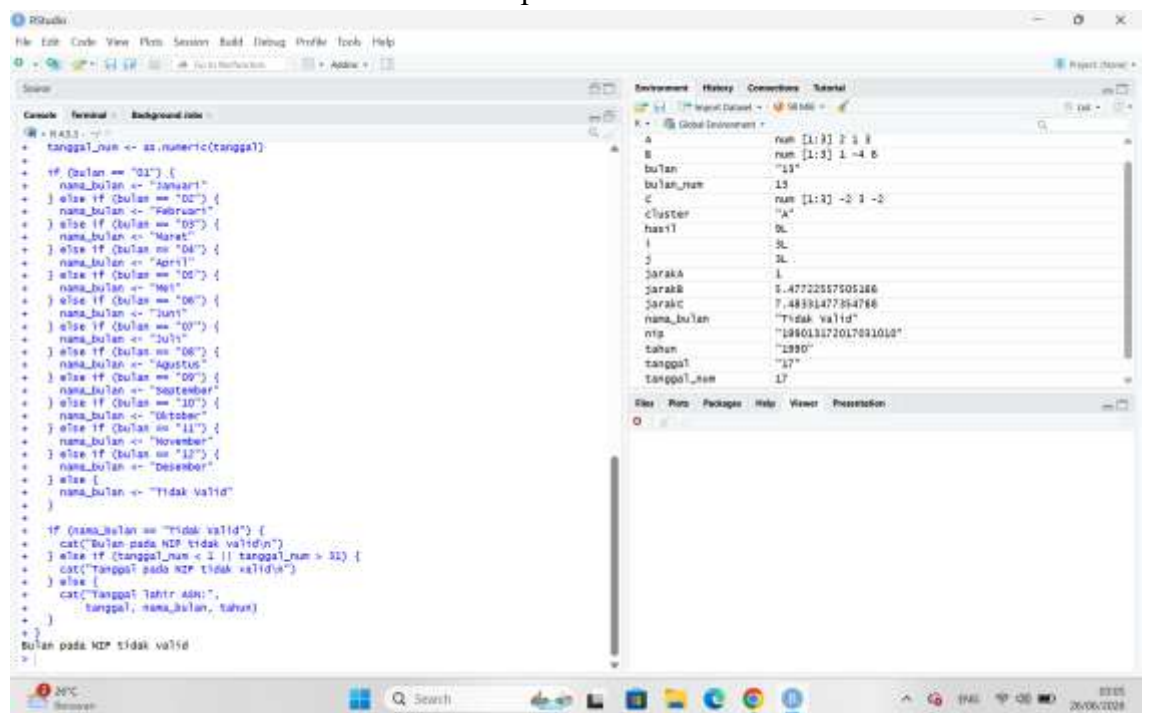
Gambar 3. Kondisi saat $D < 0$ ($a = 1, b = -2, c = 4$)

D. Pemrograman 4 :

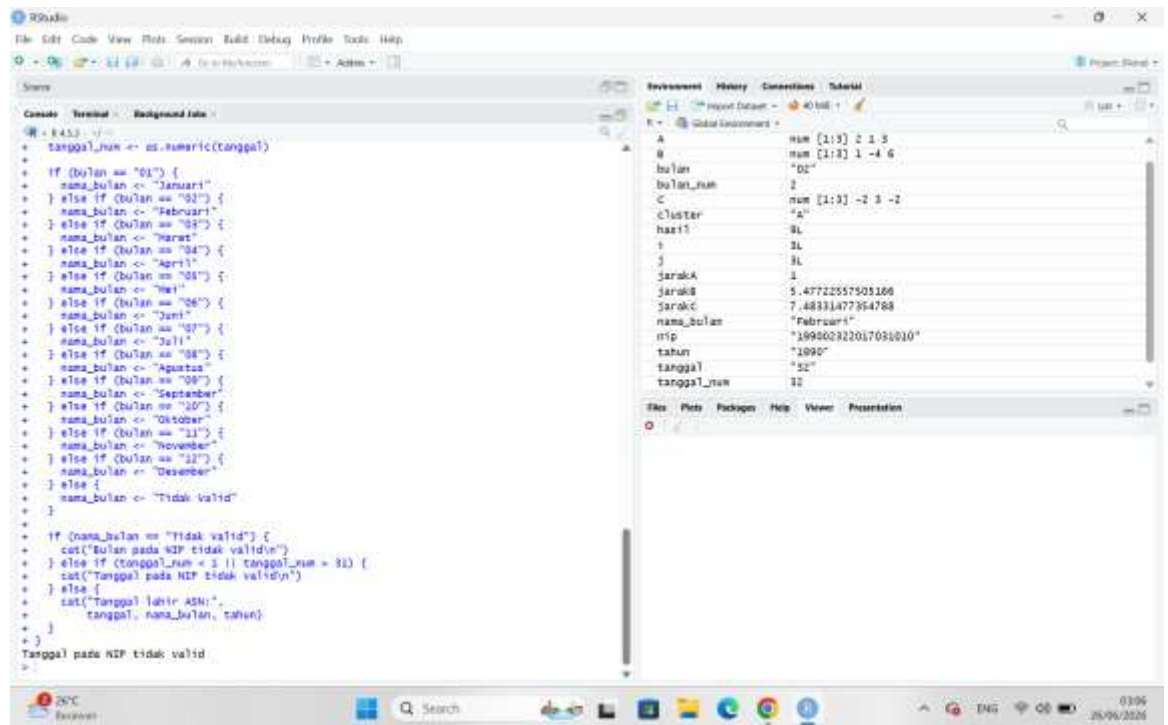
- Running R



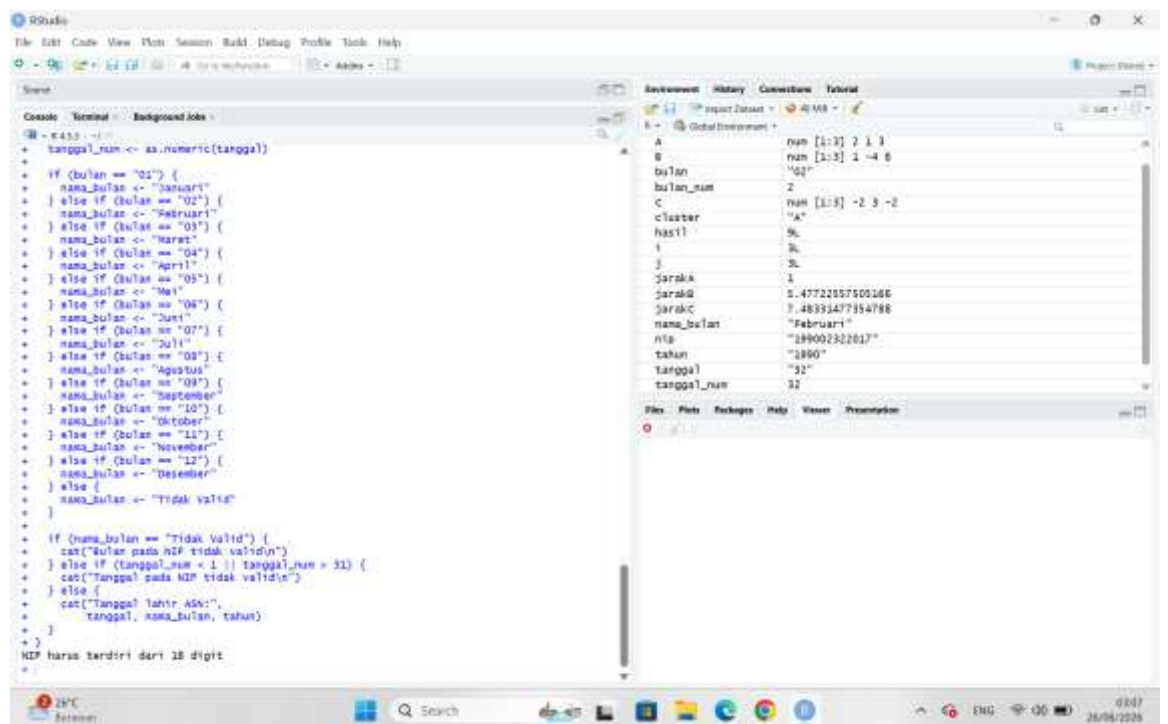
Gambar 1. Kondisi pertama NIP Valid



Gambar 2. Kondisi kedua bulan pada NIP tidak valid

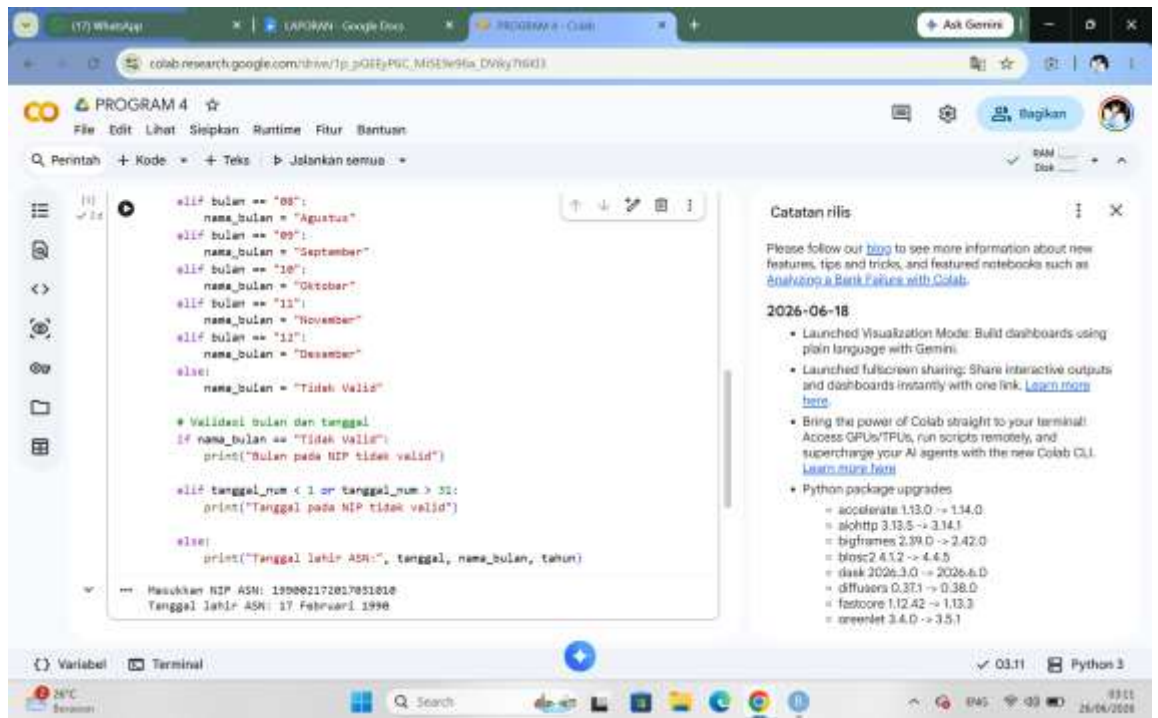


Gambar 3. Kondisi ketiga tanggal pada NIP tidak valid

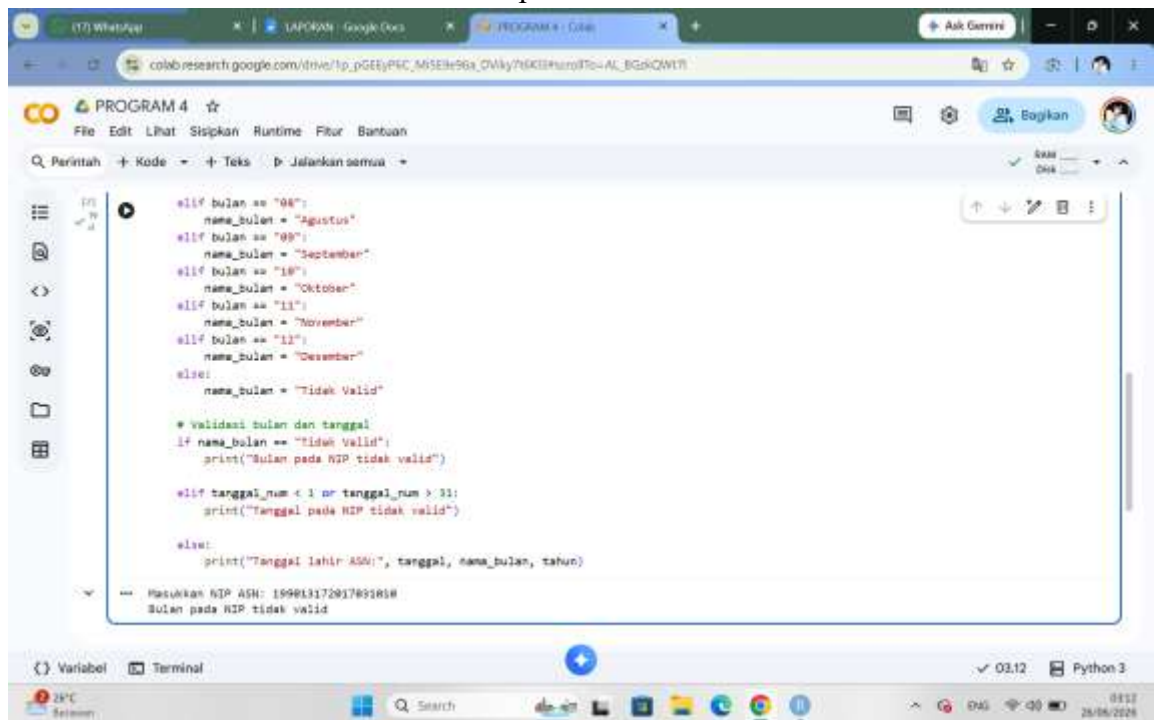


Gambar 4. Kondisi keempat NIP tidak valid

- Running Python



Gambar 1. Kondisi pertama NIP Valid



Gambar 2. Kondisi kedua bulan pada NIP tidak valid


```

elif bulan == "08":
    nama_bulan = "Agustus"
elif bulan == "09":
    nama_bulan = "September"
elif bulan == "10":
    nama_bulan = "Oktober"
elif bulan == "11":
    nama_bulan = "November"
elif bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Tidak Valid"

# Validasi bulan dan tanggal
if nama_bulan == "Tidak Valid":
    print("Bulan pada NIP tidak valid")

elif tanggal_num < 1 or tanggal_num > 31:
    print("Tanggal pada NIP tidak valid")

else:
    print("Tanggal lahir ASN:", tanggal, nama_bulan, tahun)

```

Masukkan NIP ASN: 199002312817851810
Tanggal lahir ASN: 31 Februari 1990

Gambar 3. Kondisi ketiga tanggal pada NIP tidak valid

```

elif bulan == "08":
    nama_bulan = "Agustus"
elif bulan == "09":
    nama_bulan = "September"
elif bulan == "10":
    nama_bulan = "Oktober"
elif bulan == "11":
    nama_bulan = "November"
elif bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Tidak valid"

# Validasi bulan dan tanggal
if nama_bulan == "Tidak Valid":
    print("Bulan pada NIP tidak valid")

elif tanggal_num < 1 or tanggal_num > 31:
    print("Tanggal pada NIP tidak valid")

else:
    print("Tanggal lahir ASN:", tanggal, nama_bulan, tahun)

```

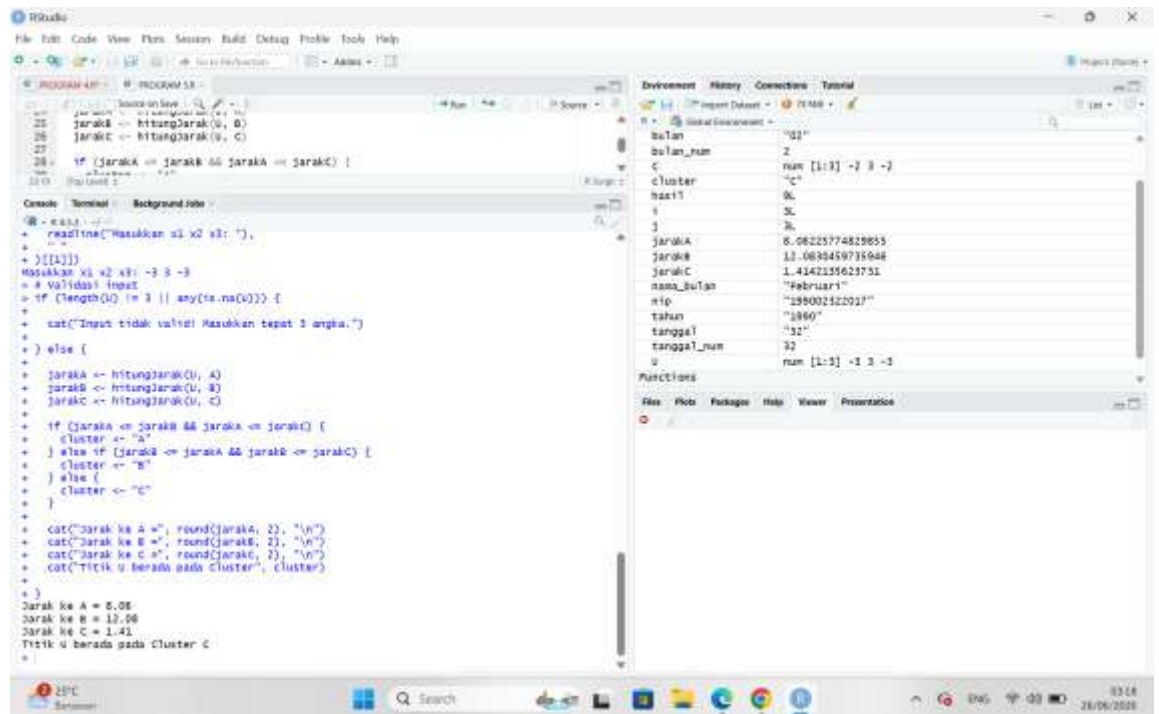
Masukkan NIP ASN: 1990021
NIP harus terdiri dari 18 digit

Gambar 4. Kondisi keempat NIP tidak valid

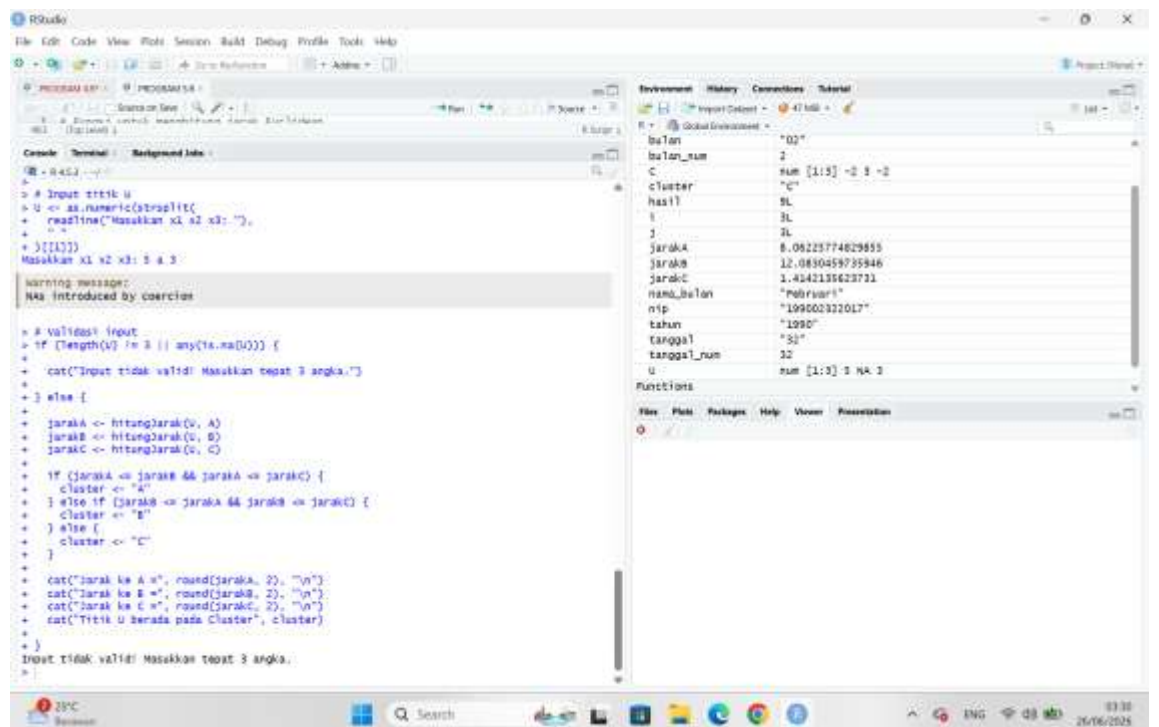
E. Pemrograman 5 :

- Running R



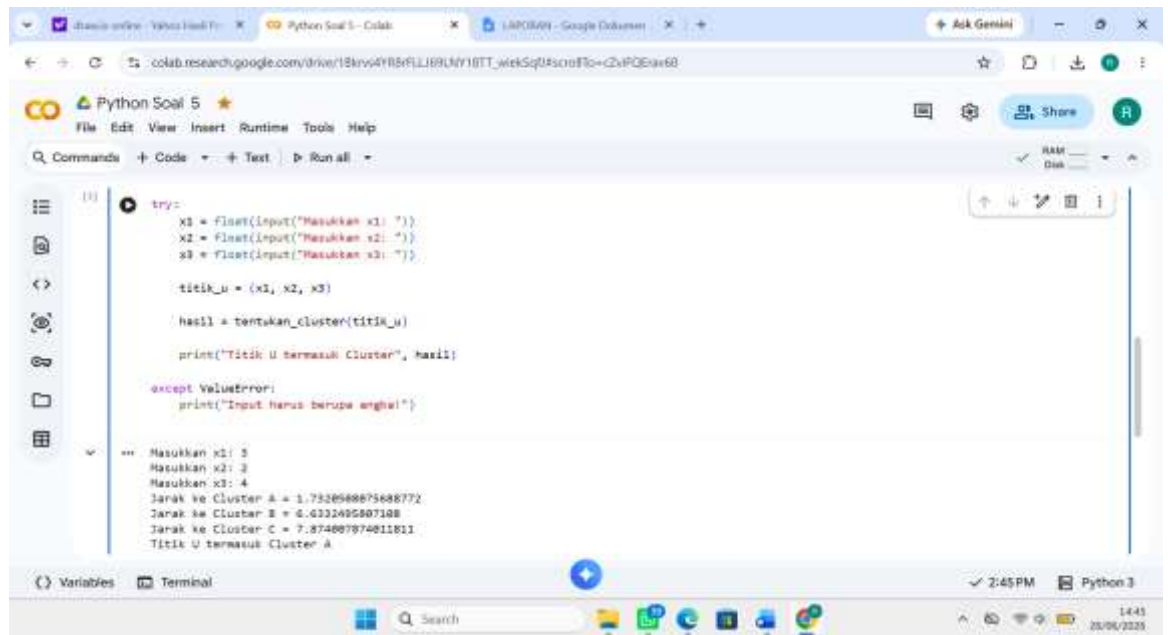


Gambar 3. Kondisi ketiga Titik U berada pada Cluster C



Gambar 4. Kondisi keempat tidak valid

- Running Python



```
try:
    x1 = float(input("Masukkan x1: "))
    x2 = float(input("Masukkan x2: "))
    x3 = float(input("Masukkan x3: "))

    titik_u = (x1, x2, x3)

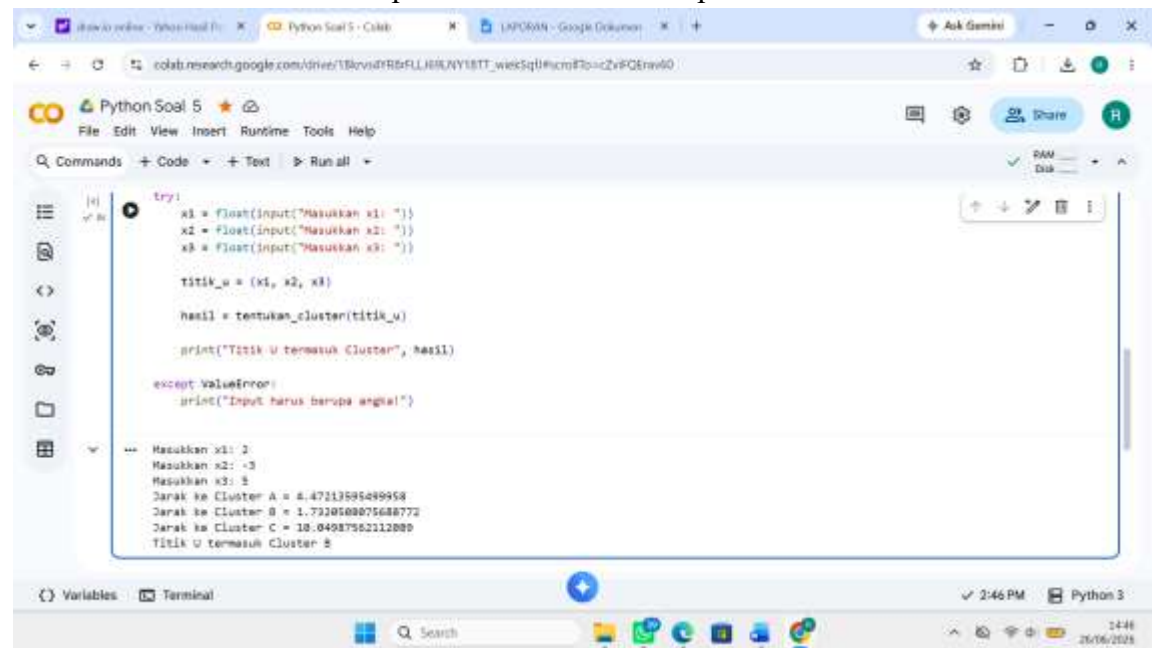
    hasil = tentukan_cluster(titik_u)

    print("Titik U termasuk Cluster", hasil)

except ValueError:
    print("Input harus berupa angka!")
```

Masukkan x1: 3
Masukkan x2: 2
Masukkan x3: 4
Jarak ke Cluster A = 1.7320508075688772
Jarak ke Cluster B = 6.032405807108
Jarak ke Cluster C = 7.874087874011811
Titik U termasuk Cluster A

Gambar 1. Kondisi pertama Titik U berada pada Cluster A.



```
try:
    x1 = float(input("Masukkan x1: "))
    x2 = float(input("Masukkan x2: "))
    x3 = float(input("Masukkan x3: "))

    titik_u = (x1, x2, x3)

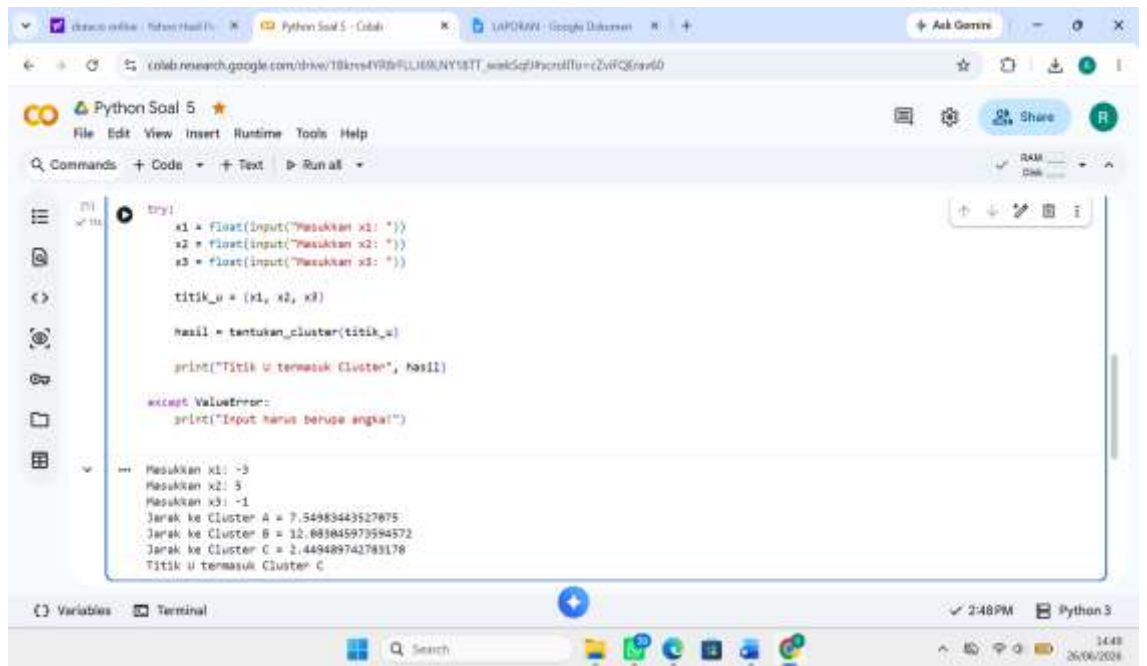
    hasil = tentukan_cluster(titik_u)

    print("Titik U termasuk Cluster", hasil)

except ValueError:
    print("Input harus berupa angka!")
```

Masukkan x1: 2
Masukkan x2: -3
Masukkan x3: 5
Jarak ke Cluster A = 4.47213595499958
Jarak ke Cluster B = 1.7320508075688772
Jarak ke Cluster C = 10.04987562112089
Titik U termasuk Cluster B

Gambar 2. Kondisi kedua Titik U berada pada Cluster B.



```
try:
    x1 = float(input("Masukkan x1: "))
    x2 = float(input("Masukkan x2: "))
    x3 = float(input("Masukkan x3: "))

    titik_u = (x1, x2, x3)

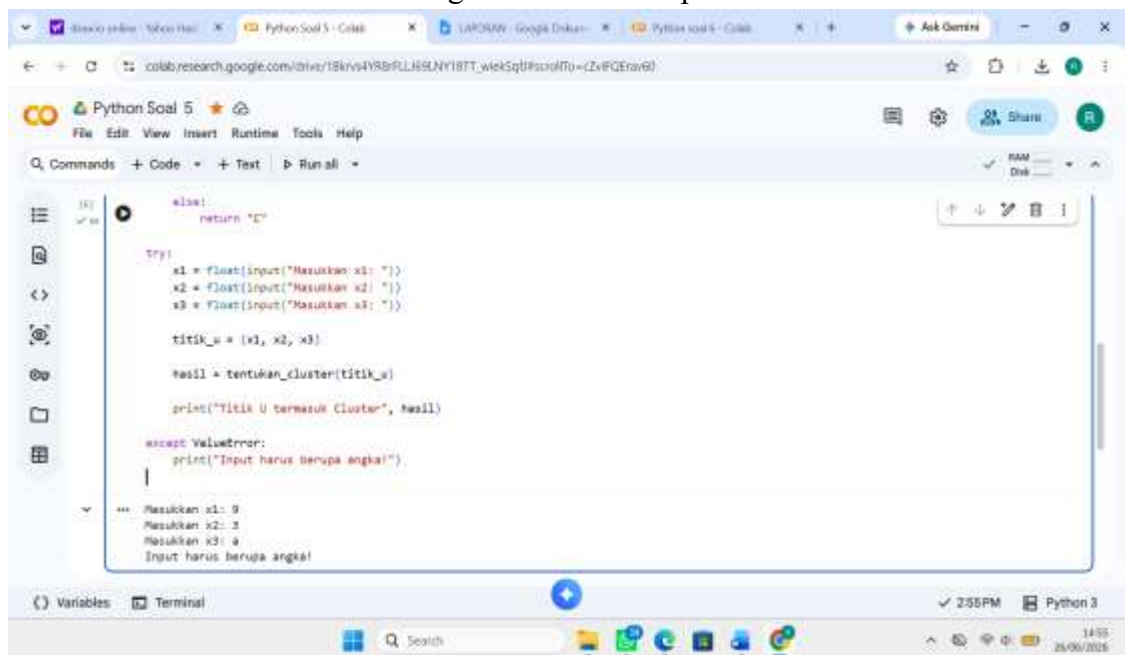
    hasil = tentukan_cluster(titik_u)

    print("Titik U termasuk Cluster", hasil)

except ValueError:
    print("Input harus berupa angka!")
```

Masukkan x1: -3
Masukkan x2: 5
Masukkan x3: -1
Jarak ke Cluster A = 7.54983443527075
Jarak ke Cluster B = 12.889845979594572
Jarak ke Cluster C = 2.449489742783178
Titik U termasuk Cluster C

Gambar 3. Kondisi ketiga Titik U berada pada Cluster C.



```
else:
    return "C"
```

```
try:
    x1 = float(input("Masukkan x1: "))
    x2 = float(input("Masukkan x2: "))
    x3 = float(input("Masukkan x3: "))

    titik_u = (x1, x2, x3)

    hasil = tentukan_cluster(titik_u)

    print("Titik U termasuk Cluster", hasil)

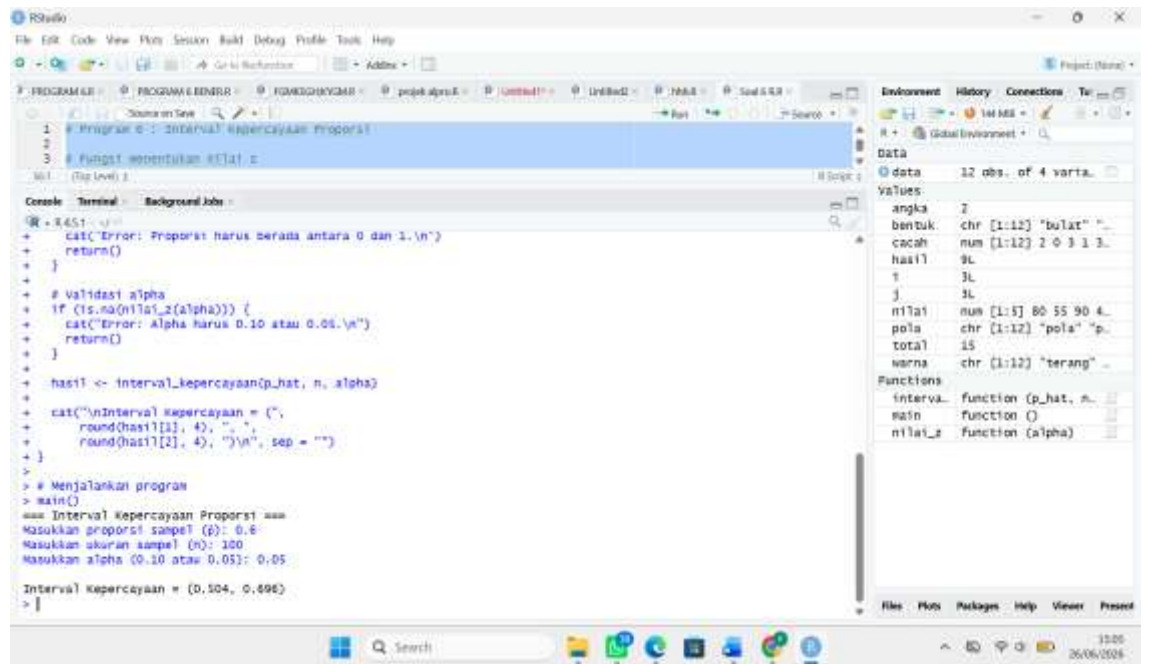
except ValueError:
    print("Input harus berupa angka!")
```

Masukkan x1: 9
Masukkan x2: 3
Masukkan x3: a
Input harus berupa angka!

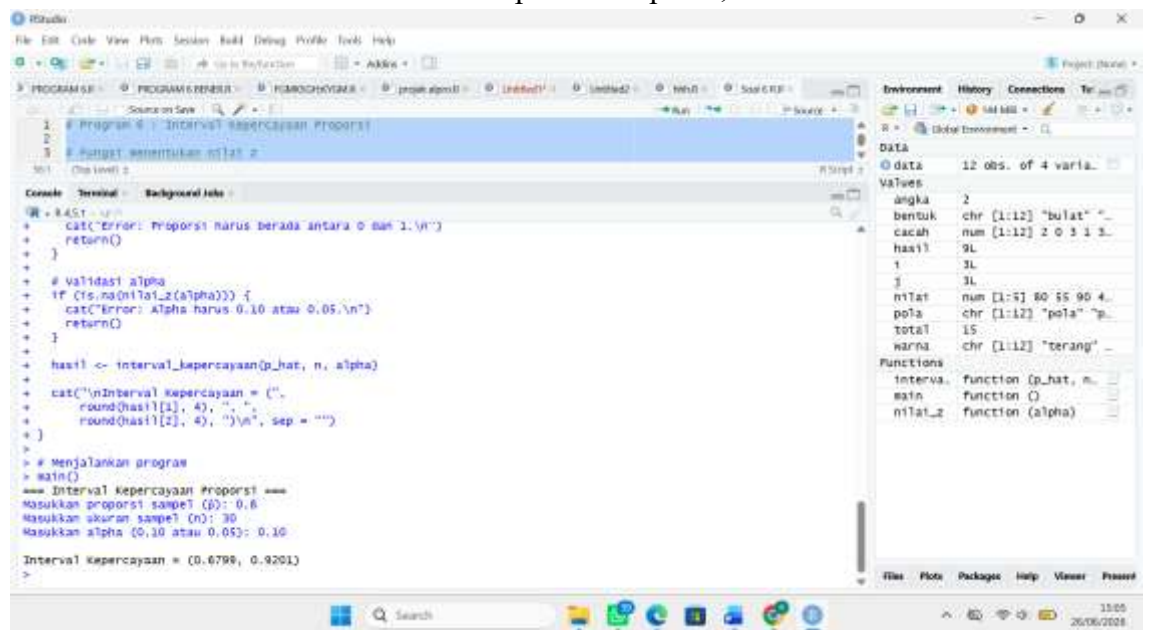
Gambar 4. Kondisi keempat tidak valid.

F. Pemrograman 6 :

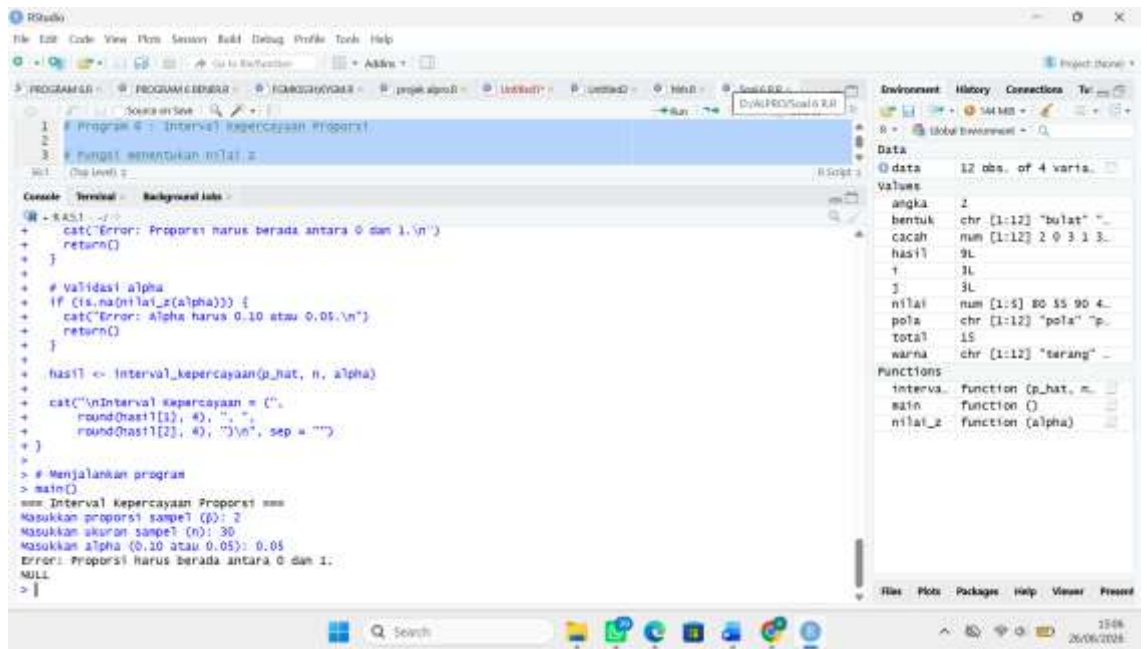
- Running R



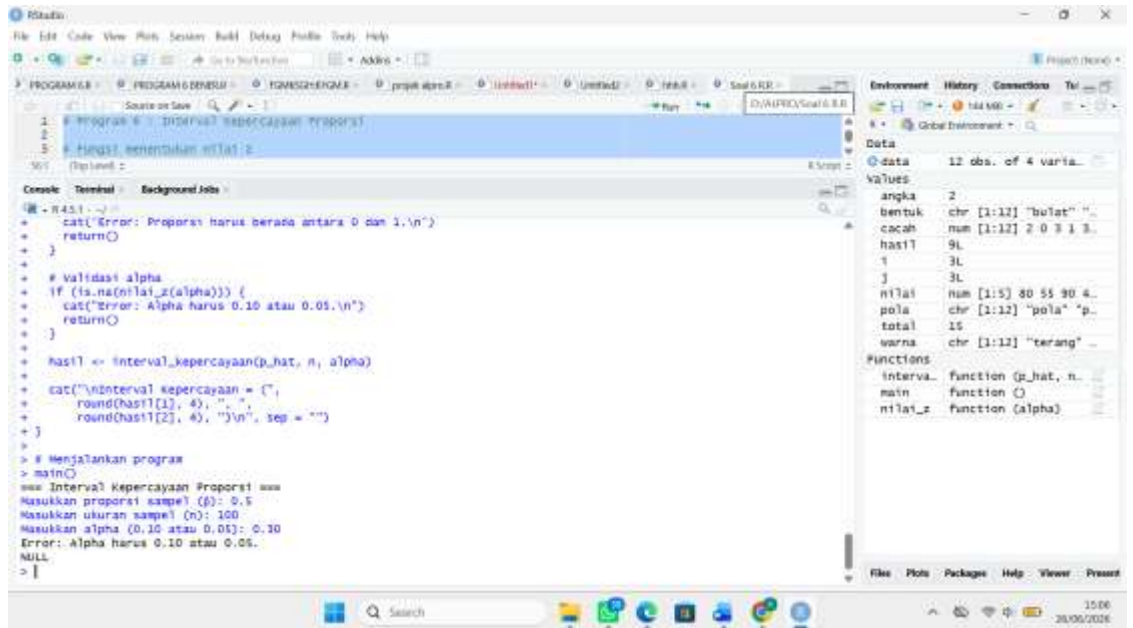
Gambar 1. Kondisi pertama alpha 0,05.



Gambar 2. Kondisi kedua alpha 0,10.

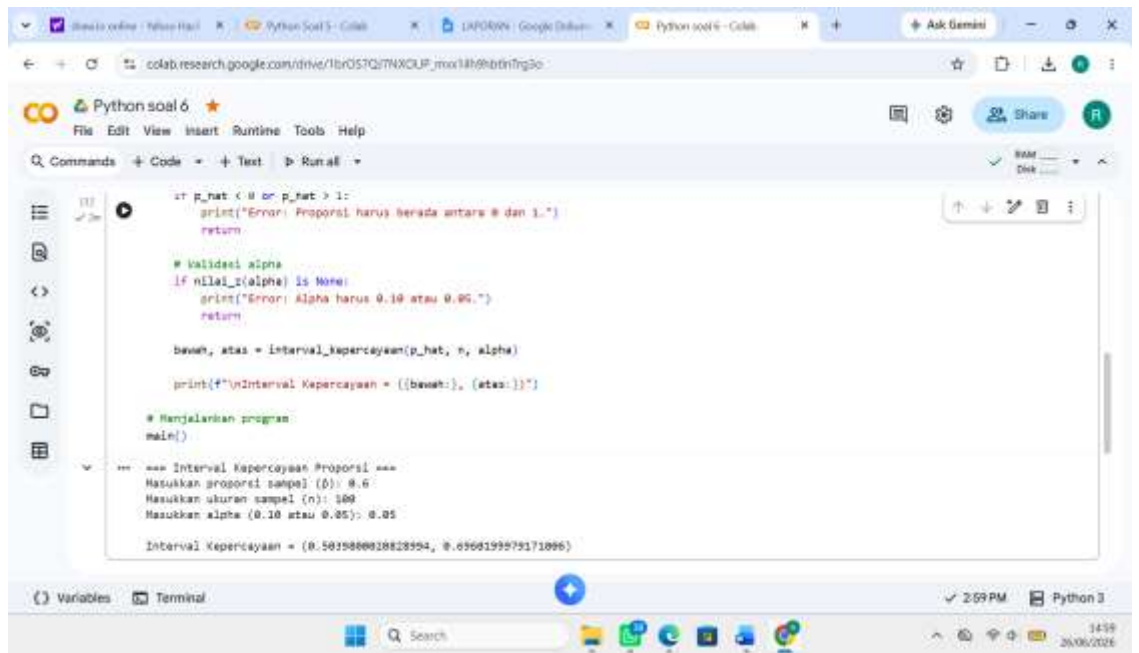


Gambar 3. Kondisi tidak valid, proporsi lebih dari 1.



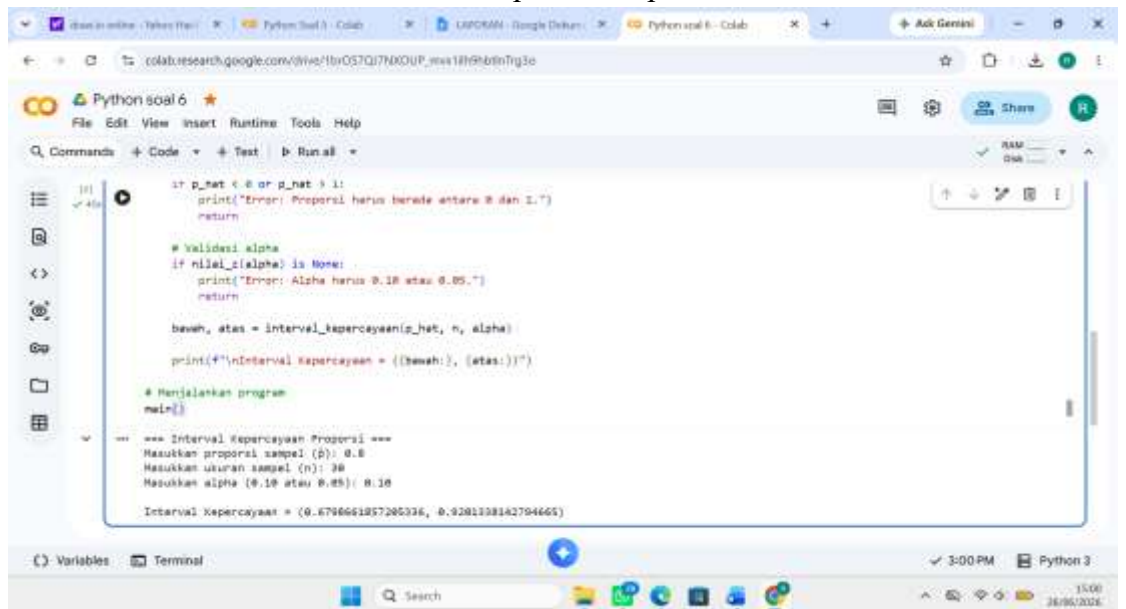
Gambar 4. Kondisi tidak valid, alpha bukan 0.05 atau 0.10.

- Running Rython



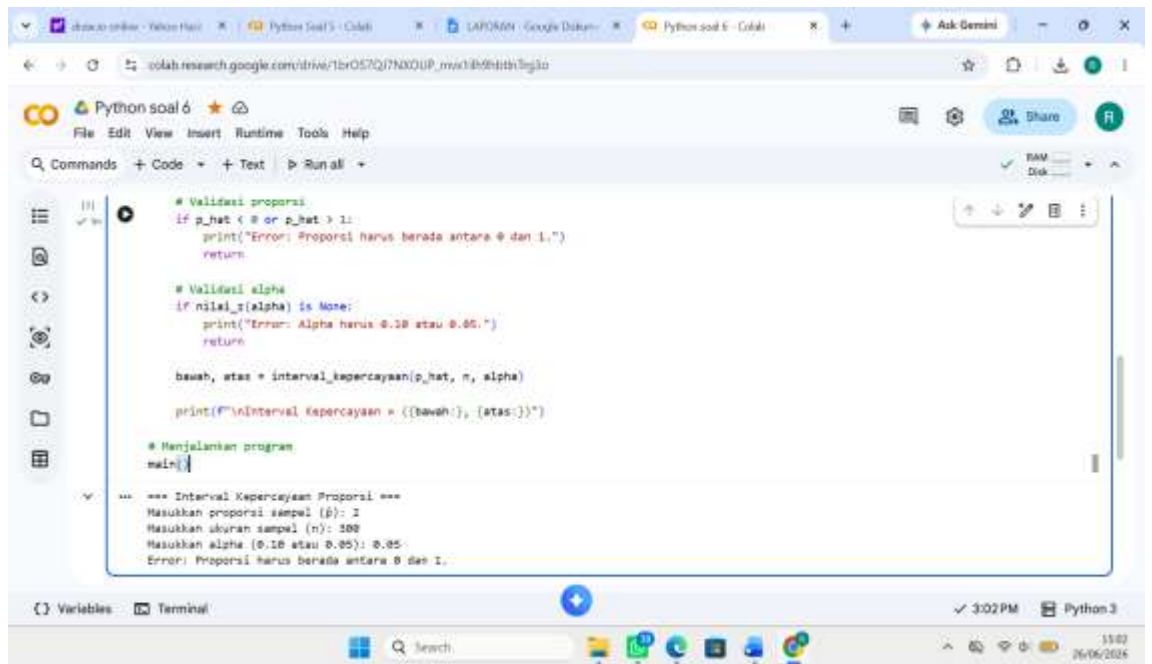
```
1 p_hat < 0 or p_hat > 1:
2     print("Error: Proporsi harus berada antara 0 dan 1.")
3     return
4
5 # Validasi alpha
6 if nilai_alpha is None:
7     print("Error: Alpha harus 0.10 atau 0.05.")
8     return
9
10 bawah, atas = interval_kepercayaan(p_hat, n, alpha)
11
12 print(f"\nInterval Kepercayaan = ({bawah:}, {atas:})")
13
14 # Menjalankan program
15 main()
16
17 === Interval Kepercayaan Proporsi ===
18 Masukkan proporsi sampel (p): 0.6
19 Masukkan ukuran sampel (n): 100
20 Masukkan alpha (0.10 atau 0.05): 0.05
21
22 Interval Kepercayaan = (0.5015888828828994, 0.6984111171171006)
```

Gambar 1. Kondisi pertama alpha 0,05.



```
1 p_hat < 0 or p_hat > 1:
2     print("Error: Proporsi harus berada antara 0 dan 1.")
3     return
4
5 # Validasi alpha
6 if nilai_alpha is None:
7     print("Error: Alpha harus 0.10 atau 0.05.")
8     return
9
10 bawah, atas = interval_kepercayaan(p_hat, n, alpha)
11
12 print(f"\nInterval Kepercayaan = ({bawah:}, {atas:})")
13
14 # Menjalankan program
15 main()
16
17 === Interval Kepercayaan Proporsi ===
18 Masukkan proporsi sampel (p): 0.6
19 Masukkan ukuran sampel (n): 100
20 Masukkan alpha (0.10 atau 0.05): 0.10
21
22 Interval Kepercayaan = (0.6796661857205336, 0.9203338142794664)
```

Gambar 2. Kondisi kedua alpha 0,10.



```

# Validasi proporsi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi harus berada antara 0 dan 1.")
    return

# Validasi alpha
if nilai_2(alpha) is None:
    print("Error: Alpha harus 0.10 atau 0.05.")
    return

bawah, atas = interval_kepercayaan(p_hat, n, alpha)

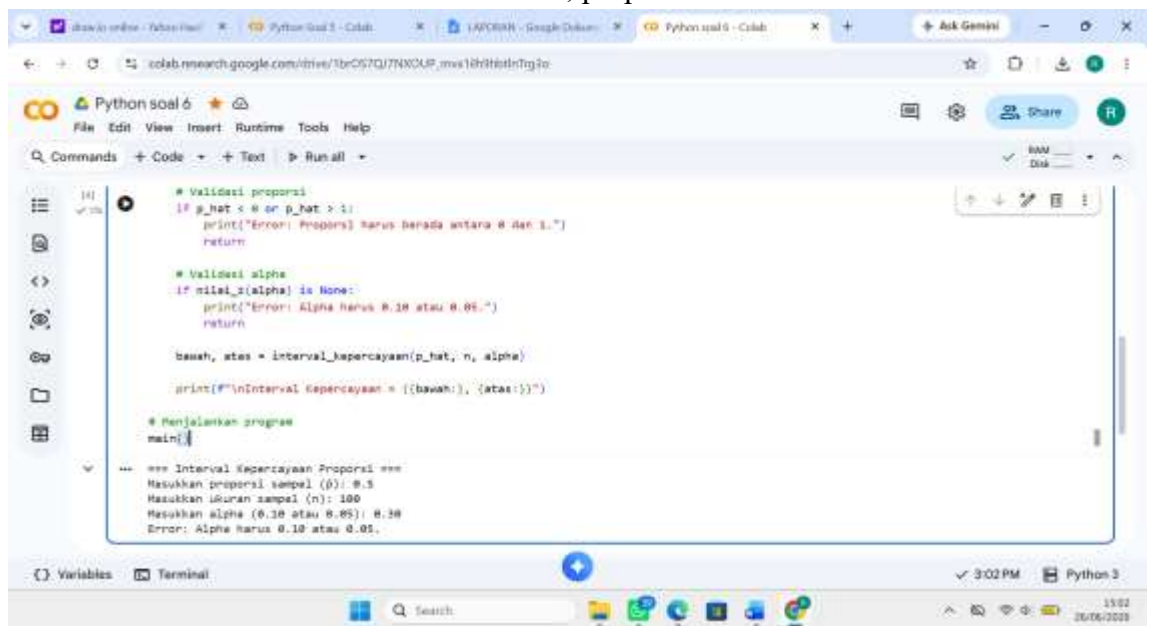
print(f"\nInterval Kepercayaan = ({bawah:}, {atas:})")

# Menjalankan program
main()

''' Interval Kepercayaan Proporsi '''
Masukkan proporsi sampel (p): 1
Masukkan ukuran sampel (n): 100
Masukkan alpha (0.10 atau 0.05): 0.05
Error: Proporsi harus berada antara 0 dan 1.

```

Gambar 3. Kondisi tidak valid, proporsi lebih dari 1.



```

# Validasi proporsi
if p_hat < 0 or p_hat > 1:
    print("Error: Proporsi harus berada antara 0 dan 1.")
    return

# Validasi alpha
if nilai_2(alpha) is None:
    print("Error: Alpha harus 0.10 atau 0.05.")
    return

bawah, atas = interval_kepercayaan(p_hat, n, alpha)

print(f"\nInterval Kepercayaan = ({bawah:}, {atas:})")

# Menjalankan program
main()

''' Interval Kepercayaan Proporsi '''
Masukkan proporsi sampel (p): 0.5
Masukkan ukuran sampel (n): 100
Masukkan alpha (0.10 atau 0.05): 0.10
Error: Alpha harus 0.10 atau 0.05.

```

Gambar 4. Kondisi tidak valid, alpha bukan 0.05 atau 0.10.

6.3 Interpretasi Output

A. Pemrograman 1 :

Pengujian program dilakukan menggunakan tiga kondisi yang berbeda untuk memastikan program dapat berjalan dengan baik sesuai tujuan yang diharapkan.

Kondisi pertama dilakukan dengan memasukkan satu kalimat, yaitu *"Kelompok kami sedang mengerjakan tugas algoritma pemrograman."*. Hasil yang diperoleh menunjukkan bahwa program berhasil menghitung **7 kata** dan **1 kalimat**. Hal ini menunjukkan bahwa program dapat mengenali setiap kata yang dipisahkan oleh spasi serta menghitung satu tanda titik sebagai penanda akhir kalimat.

Kondisi kedua dilakukan dengan memasukkan beberapa kalimat dalam satu paragraf, yaitu *"Kami suka dengan tugas besar kali ini karena ilmu yang dipelajari dalam setiap soal sangat bermanfaat untuk kedepannya. Apalagi di era yang sudah canggih ini kita perlu untuk terus belajar teknologi agar tidak ketinggalan. Selain itu video demo yang kami buat semoga dapat bermanfaat bagi orang yang menonton. Terimakasih sekian dari kelompok kami."*. Hasil pengujian menunjukkan bahwa program menghasilkan **52 kata** dan **4 kalimat**. Hasil ini membuktikan bahwa program mampu mengolah teks yang lebih panjang dan menghitung jumlah kata serta jumlah kalimat dengan benar berdasarkan banyaknya tanda titik yang terdapat pada paragraf.

Kondisi ketiga dilakukan tanpa memasukkan teks apa pun (input kosong). Pada kondisi ini program menghasilkan **0 kata** dan **0 kalimat**. Hal ini menunjukkan bahwa program tetap dapat berjalan meskipun tidak ada input yang diberikan. Karena tidak terdapat kata maupun tanda titik pada input kosong, maka jumlah kata dan jumlah kalimat yang dihitung adalah nol.

Berdasarkan ketiga pengujian tersebut, dapat disimpulkan bahwa program mampu menghitung jumlah kata dan jumlah kalimat dengan baik pada berbagai kondisi input, baik berupa satu kalimat, beberapa kalimat, maupun input kosong.

B. Pemrograman 2 :

Pada Program 2, pengujian dilakukan dengan menjalankan program untuk menampilkan empat buah string yang telah disimpan dalam variabel K1, K2, K3, dan K4. Hasil yang diperoleh menunjukkan bahwa seluruh string berhasil ditampilkan sesuai dengan format yang telah ditentukan pada soal.

Output pertama menampilkan kalimat **"Saya tak 'kan menyerah."** yang menunjukkan bahwa program dapat menampilkan tanda petik tunggal (') sebagai bagian dari teks. Output kedua menampilkan kalimat **"Ia berkata, "Aku menyayangimu.""** yang membuktikan bahwa program dapat menampilkan tanda petik ganda (") dengan benar. Selanjutnya, output ketiga menampilkan kalimat **""Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!""** yang mengandung kombinasi tanda petik tunggal dan tanda petik ganda, sehingga menunjukkan bahwa penggunaan karakter khusus dalam string telah ditangani dengan baik. Output terakhir menampilkan kalimat **"Surat keputusan itu bernomor 62/UN.34/19/2023."** sesuai dengan yang telah ditentukan.

Berdasarkan hasil pengujian tersebut, dapat disimpulkan bahwa program berhasil membuat dan menampilkan seluruh string sesuai dengan format yang diminta pada soal. Selain itu, program juga mampu menampilkan karakter khusus seperti tanda petik tunggal dan tanda petik ganda tanpa menimbulkan kesalahan, sehingga tujuan program telah tercapai dengan baik.

C. Pemrograman 3 :

Kondisi pertama dilakukan dengan memasukkan nilai koefisien dari persamaan kuadrat yaitu nilai $a = 1$, $b = -5$ dan $c = 6$. Hasil yang di peroleh adalah akar real berbeda dengan nilai $x_1 = 3$ dan $x_2 = 2$. Hasil ini membuktikan bahwa program mampu berjalan dengan baik untuk menghitung akar kuadrat dengan nilai diskriminan lebih dari 0

Kondisi kedua dilakukan dengan memasukkan nilai koefisien dari persamaan kuadrat yaitu nilai $a = 1$, $b = -4$ dan $c = 4$. Hasil yang di peroleh adalah akar real kembar dengan nilai $x_1 = x_2 = 2$. Hasil ini membuktikan bahwa program mampu berjalan dengan baik untuk menghitung akar kuadrat dengan nilai diskriminan sama dengan 0

Kondisi ketiga dilakukan dengan memasukkan nilai koefisien dari persamaan kuadrat yaitu nilai $a = 1$, $b = -2$ dan $c = 4$. Hasil yang di peroleh adalah “persamaan ini memiliki akar-akar imajiner”. Hasil ini membuktikan bahwa program mampu berjalan dengan baik untuk menghitung akar kuadrat dengan nilai diskriminan kurang dari 0

D. Pemrograman 4 :

Output dari program ini menunjukkan tanggal lahir ASN yang berhasil diperoleh dari 8 digit pertama NIP. Jika NIP yang dimasukkan valid, program akan menampilkan tanggal lahir dalam format tanggal nama_bulan tahun. Sebagai contoh, jika pengguna memasukkan NIP 199002172019031010, maka program menghasilkan output “Tanggal lahir ASN: 17 Februari 1990”. Hal ini berarti program berhasil mengambil tahun 1990, bulan 02 yang dikonversi menjadi Februari, serta tanggal 17 dari NIP tersebut.

Selain menampilkan hasil tanggal lahir, program juga mampu memberikan pesan kesalahan apabila input tidak sesuai ketentuan. Jika panjang NIP tidak 18 digit, program menampilkan pesan “NIP harus terdiri dari 18 digit”. Jika kode bulan tidak berada pada rentang 01–12, program akan menampilkan “Bulan pada NIP tidak valid”. Sedangkan jika nilai tanggal berada di luar rentang 1–31, program akan menampilkan “Tanggal pada NIP tidak valid”. Dengan demikian, output program tidak hanya menunjukkan hasil akhir, tetapi juga berfungsi sebagai bentuk validasi untuk memastikan bahwa data NIP yang dimasukkan sesuai format yang benar.

E. Pemrograman 5 :

Berdasarkan hasil pengujian yang telah dilakukan, program mampu mengklasifikasikan titik ke dalam cluster yang sesuai berdasarkan jarak Euclidean terhadap pusat Cluster A, Cluster B, dan Cluster C. Seluruh skenario uji menunjukkan status berhasil, sehingga dapat disimpulkan bahwa logika perhitungan jarak dan proses penentuan cluster telah berjalan dengan baik.

Pada skenario uji pertama, titik yang dimasukkan berada lebih dekat dengan pusat Cluster A, sehingga program menampilkan hasil bahwa titik U termasuk Cluster A. Hal ini menunjukkan bahwa proses perhitungan jarak ke masing-masing pusat cluster dan proses perbandingan jarak telah dilakukan dengan benar.

Pada skenario uji kedua, titik yang dimasukkan memiliki jarak paling kecil terhadap pusat Cluster B, sehingga program berhasil mengklasifikasikan titik tersebut sebagai anggota Cluster B. Hasil ini membuktikan bahwa program mampu membedakan cluster berdasarkan nilai jarak terdekat.

Pada skenario uji ketiga, titik yang dimasukkan lebih dekat dengan pusat Cluster C, sehingga program menampilkan hasil bahwa titik U berada pada Cluster C. Dengan demikian, seluruh kemungkinan hasil klasifikasi, yaitu Cluster A, Cluster B, dan Cluster C, telah berhasil diuji dan memberikan hasil yang sesuai.

Selain pengujian terhadap data yang valid, dilakukan pula pengujian terhadap beberapa kondisi input yang tidak sesuai. Pada skenario keempat, ketika jumlah koordinat yang dimasukkan kurang dari tiga angka, program menampilkan pesan "Input tidak valid! Masukkan tepat 3 angka." sehingga proses klasifikasi tidak dilanjutkan. Hal ini menunjukkan bahwa program mampu memvalidasi jumlah koordinat yang diberikan oleh pengguna.

Pada skenario kelima, pengguna memasukkan lebih dari tiga angka sebagai koordinat titik. Program kembali menampilkan pesan kesalahan yang sama, sehingga hanya input yang terdiri dari tepat tiga koordinat yang dapat diproses. Mekanisme ini mencegah kesalahan dalam proses perhitungan jarak.

Selanjutnya, pada skenario keenam dilakukan pengujian dengan memasukkan karakter huruf sebagai salah satu nilai koordinat. Program berhasil mendeteksi bahwa input bukan berupa angka dan menampilkan pesan kesalahan sesuai yang diharapkan. Dengan demikian, program telah memiliki mekanisme validasi yang mampu menangani kesalahan input dari pengguna.

Secara keseluruhan, hasil pengujian menunjukkan bahwa program tidak hanya mampu menentukan cluster yang benar untuk data yang valid, tetapi juga dapat menangani berbagai kondisi input yang tidak valid melalui proses validasi. Hal ini membuktikan bahwa program telah memenuhi kebutuhan soal dan dapat digunakan dengan baik untuk mengklasifikasikan titik ke dalam cluster berdasarkan jarak terdekat.

F. Pemrograman 6 :

Berdasarkan hasil pengujian yang telah dilakukan, seluruh skenario uji menunjukkan bahwa program dapat berjalan sesuai dengan fungsi dan tujuan yang telah dirancang. Program mampu menghitung interval kepercayaan proporsi dengan benar ketika menerima input yang valid, serta mampu melakukan validasi terhadap input yang tidak sesuai dengan ketentuan.

Pada skenario uji pertama, dengan input ($\hat{p}=0,6$), ($n=100$), dan ($\alpha=0,05$), program berhasil menghitung interval kepercayaan sebesar (0,504; 0,696). Hasil tersebut menunjukkan bahwa dengan tingkat kepercayaan 95%, proporsi populasi diperkirakan berada pada rentang tersebut. Hal ini menunjukkan bahwa perhitungan margin of error dan penentuan batas bawah maupun batas atas telah dilakukan dengan benar.

Pada skenario uji kedua, digunakan input ($\hat{p}=0,8$), ($n=30$), dan ($\alpha=0,10$). Program berhasil menghasilkan interval kepercayaan sebesar (0,6799; 0,9201). Hasil ini menunjukkan bahwa program dapat menyesuaikan nilai z berdasarkan tingkat

signifikansi yang dipilih, sehingga perhitungan interval kepercayaan tetap sesuai dengan rumus yang digunakan.

Pada skenario uji ketiga, nilai proporsi yang dimasukkan adalah 2, sehingga berada di luar rentang yang diperbolehkan, yaitu $0 \leq \hat{p} \leq 1$. Program mendeteksi kesalahan tersebut dan menampilkan pesan "Error: Proporsi harus berada antara 0 dan 1.", kemudian menghentikan proses perhitungan. Hal ini membuktikan bahwa proses validasi input proporsi telah berfungsi dengan baik sehingga dapat mencegah terjadinya perhitungan yang tidak valid.

Pada skenario uji keempat, nilai alpha yang dimasukkan adalah 0,30, sedangkan program hanya menerima nilai 0,10 atau 0,05. Program kemudian menampilkan pesan "Error: Alpha harus 0.10 atau 0.05." dan tidak melanjutkan proses perhitungan. Hal ini menunjukkan bahwa validasi terhadap nilai alpha juga telah berjalan sesuai dengan ketentuan yang ditetapkan.

Secara keseluruhan, seluruh skenario uji memperoleh status berhasil. Hal ini menunjukkan bahwa program tidak hanya mampu menghitung interval kepercayaan proporsi secara tepat untuk data yang valid, tetapi juga mampu menangani kesalahan input melalui mekanisme validasi, sehingga program menjadi lebih andal dan sesuai dengan spesifikasi yang diberikan pada soal.

BAB VII

PENUTUP

7.1 Kesimpulan

Berdasarkan hasil pelaksanaan project Algoritma Pemrograman yang telah dilakukan, dapat disimpulkan bahwa seluruh tujuan project telah tercapai dengan baik. Enam studi kasus yang dikerjakan berhasil dianalisis, dirancang dalam bentuk pseudocode dan flowchart, kemudian diimplementasikan menggunakan bahasa pemrograman Python dan R sesuai dengan kebutuhan masing-masing permasalahan. Setiap program mampu menghasilkan output yang sesuai dengan tujuan yang telah ditetapkan, mulai dari pengolahan teks, pembuatan objek string, perhitungan akar persamaan kuadrat, penentuan tanggal lahir berdasarkan NIP ASN, klasifikasi titik ke dalam cluster menggunakan jarak Euclidean, hingga perhitungan interval kepercayaan proporsi.

Berdasarkan hasil pengujian, seluruh program dapat berjalan dengan baik pada berbagai skenario input, baik input normal maupun beberapa kondisi tidak valid yang telah ditentukan. Hal tersebut menunjukkan bahwa logika algoritma, penggunaan struktur kontrol, serta proses validasi input telah diterapkan dengan baik sehingga program mampu memberikan hasil yang benar sesuai spesifikasi. Selain meningkatkan pemahaman mengenai konsep dasar algoritma dan pemrograman, project ini juga memberikan pengalaman dalam menyusun dokumentasi program, melakukan pengujian, serta mengimplementasikan solusi yang sama pada dua bahasa pemrograman yang berbeda.

7.2 Saran

Untuk pengembangan selanjutnya, program yang telah dibuat masih dapat disempurnakan dengan menambahkan validasi input yang lebih lengkap agar mampu menangani lebih banyak kondisi khusus dan meminimalkan kesalahan saat proses eksekusi. Selain itu, efisiensi kode dapat ditingkatkan dengan memanfaatkan fungsi atau struktur program yang lebih modular sehingga lebih mudah dipelajari, dipelihara, dan dikembangkan.

Di sisi lain, pengembangan fitur juga dapat dilakukan sesuai dengan kebutuhan masing-masing program, misalnya menambahkan dukungan terhadap lebih banyak variasi input, meningkatkan tampilan output agar lebih informatif, serta memperluas cakupan permasalahan yang dapat diselesaikan. Dengan demikian, program yang dihasilkan tidak hanya memenuhi kebutuhan tugas perkuliahan, tetapi juga memiliki nilai praktis yang lebih tinggi untuk diterapkan pada permasalahan nyata.

LAMPIRAN

LINK GIT HUB :

<https://github.com/avonafebrary/KELOMPOK-C-ALGORITMA-PEMROGRAMAN-KELAS-A>

LINK VIDIO DEMOSTRASI :

https://drive.google.com/drive/folders/1fDZK7mhgm-em0eGm0wTo_Sv5OOKLe3ky

LINK AI PROGRAM 1 :

https://chatgpt.com/s/t_6a3e16c90d688191bc080aa16ca87979
https://chatgpt.com/s/t_6a3e173d80e48191996610633179d4dd

LINK AI PROGRAM 2 :

https://chatgpt.com/s/t_6a3e16ee29e08191a2cd82190b825733
https://chatgpt.com/s/t_6a3e17788d988191b35249aece76aac7

LINK AI PROGRAM 3 :

<https://chatgpt.com/c/6a324212-99e8-83ec-b9af-6df5924ce351>

LINK AI PROGRAM 4

<https://chatgpt.com/share/6a3e43ce-807c-83ec-9749-2d67cc52fd42>

LINK AI PROGRAM 5

<https://chatgpt.com/share/6a3e644e-7560-83ec-8f09-8f4b4d3b87d0>

LINK AI PROGRAM 6

<https://chatgpt.com/share/6a3e64a7-9474-83ec-8748-306117570833>